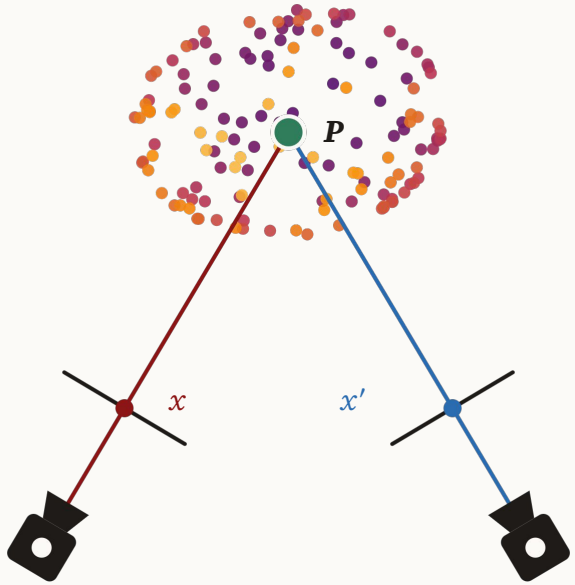
LECTURE · 3D COMPUTER VISION

From Pixels to Points

How a 3D point in the world becomes a pixel — and how we recover it back.

[Instructor name] · [Course, Session __] · [Date]

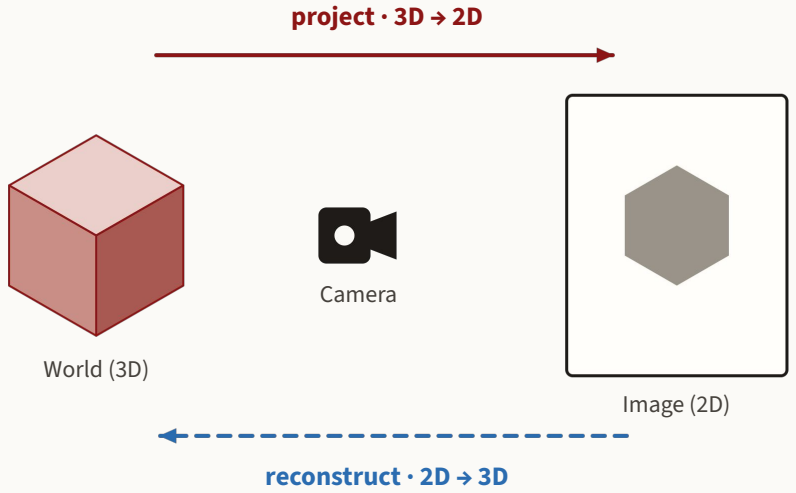


What is 3D vision?

3D vision recovers the geometry of the world — where things are, how far away, what shape — from images.

- A photo flattens a 3D world onto a 2D sensor
- Depth is the dimension that gets lost
- 3D vision is the art of getting it back

Humans do this effortlessly: two eyes, motion, and a lifetime of experience.



Where 3D vision shows up



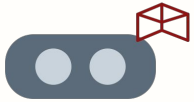
Autonomous driving

Distance to every car, cyclist and curb



Robotics

Grasping, navigation, avoiding obstacles



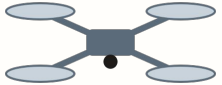
AR / VR

Anchor virtual objects to real surfaces



Mapping

City-scale 3D maps from photos and scans



Drones

Inspection, landing, terrain following

Common thread: every one of them needs to know *where* things are in 3D — not just *what* they are.

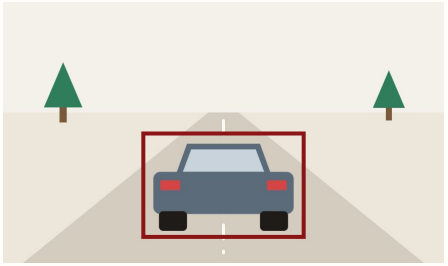
2D perception vs 3D perception



Classification

What is in the image?

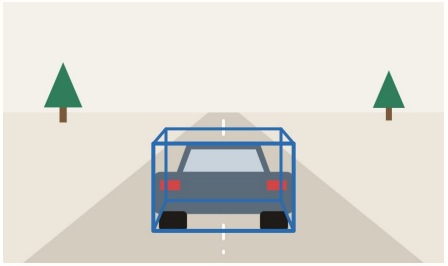
Output: “car”



2D detection

What, and where in the image?

Output: a pixel box



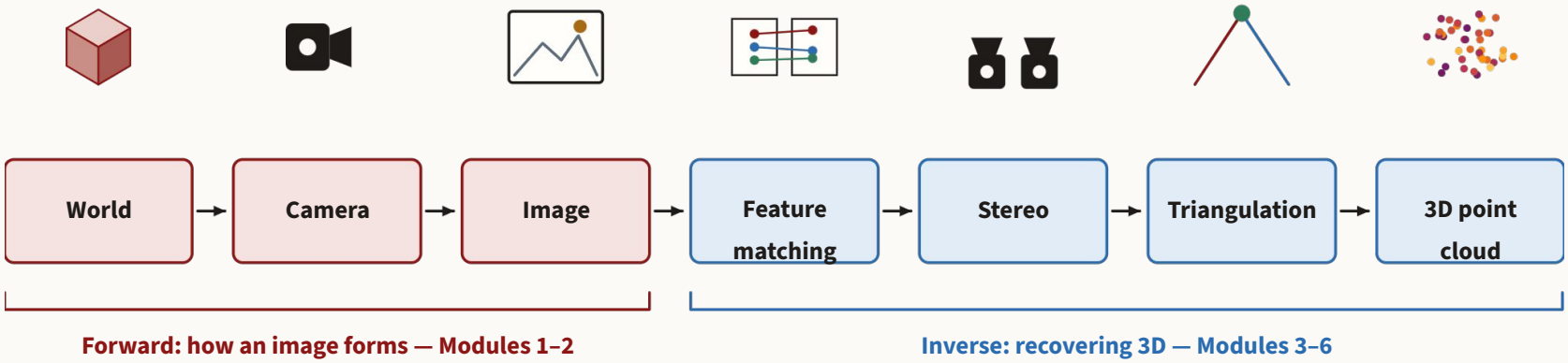
3D localization

What, and where in the world?

Output: position, size, heading

Each step adds geometry. The last one needs **depth** — which a single image does not give you directly.

The complete pipeline



Today: walk left to right to understand the camera, then right to left to undo it.

Roadmap of today's lecture

One story in eight chapters — each module answers one step of **pixel ↔ point**.

#	Module	The question it answers
0	Why 3D vision matters	Why do we need depth at all?
1	Camera projection	How does a 3D point become a pixel?
2	Camera calibration	How do we find K, R and t?
3	Homography	What if everything lies on a plane?
4	Stereo vision	How do two cameras recover depth?
5	Epipolar geometry	How do we find matches quickly?
6	Triangulation	How do we recover the 3D point?
7	Putting it together	What does the full pipeline look like?

MODULE 1

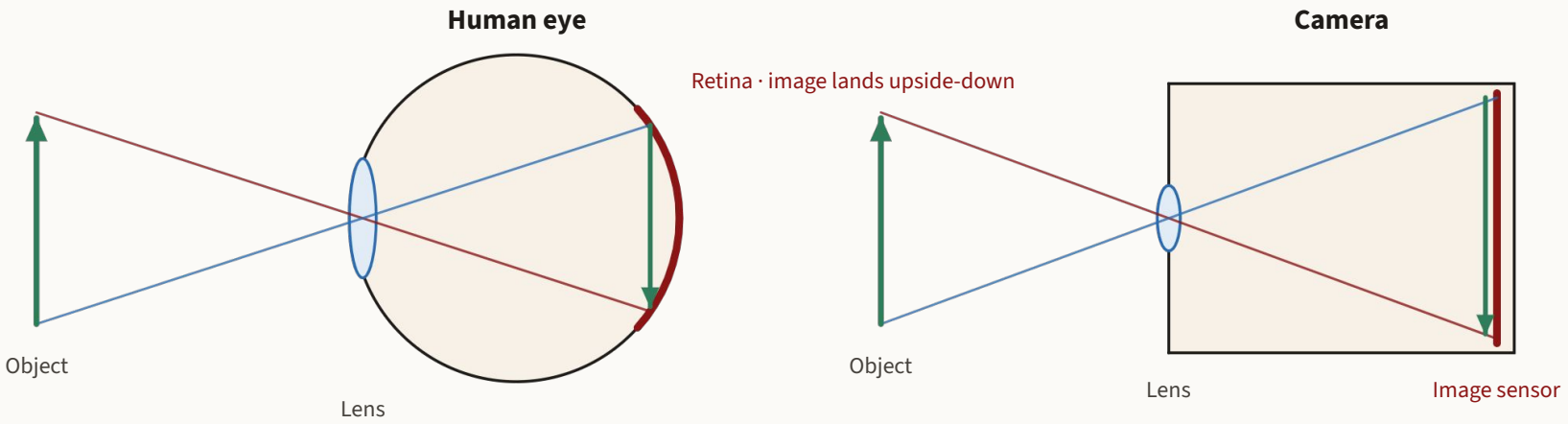
Camera Projection

Goal — how does a 3D point become a pixel?

01

Why 3D | **Projection** | Calibration | Homography | Stereo | Epipolar | Triangulation | Pipeline

Your eye is a camera



Lens ↔ Lens
Bends light so it comes to a focus

Retina ↔ Image sensor
Records where the light lands

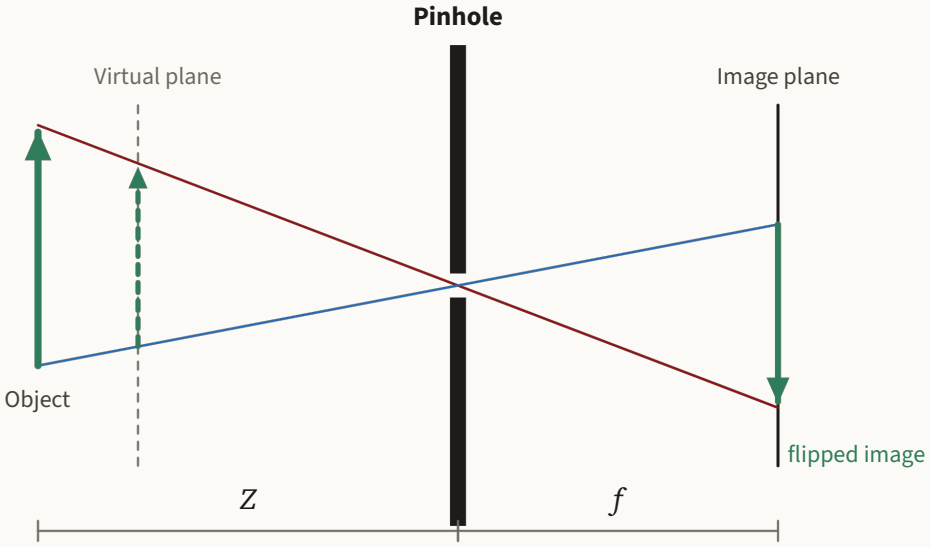
Seeing ↔ Projection
A 3D scene becomes a flat 2D image

The pinhole camera

Block every ray except those through one tiny hole.

- Each scene point → exactly one image point
- The image comes out upside-down
- Image size depends on *f*, the hole-to-sensor distance
- Trick: imagine a **virtual plane** in front of the hole — upright image, same math

> **Think**
> What happens to the image if the hole gets bigger? Smaller?



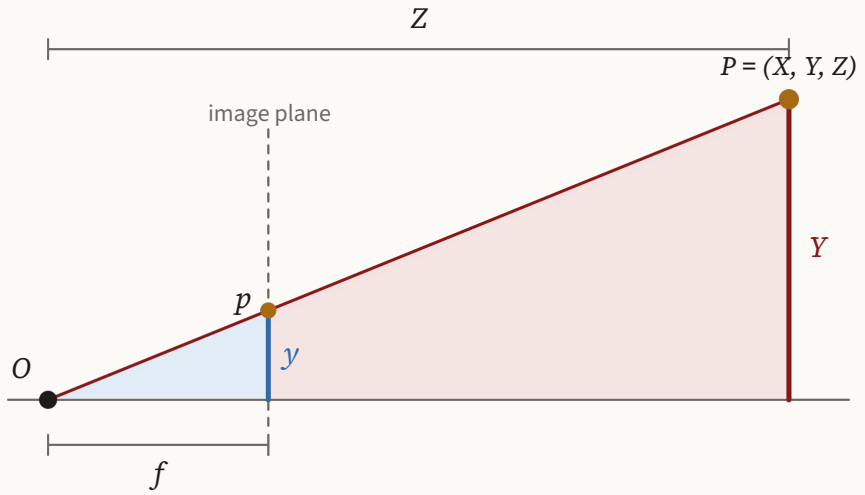
Similar triangles do all the work

Two triangles share the angle at O, so their sides are in proportion:

$$y / f = Y / Z$$

$$x = f \cdot X / Z \quad , \quad y = f \cdot Y / Z$$

Perspective is one division by depth Z. No matrices yet.

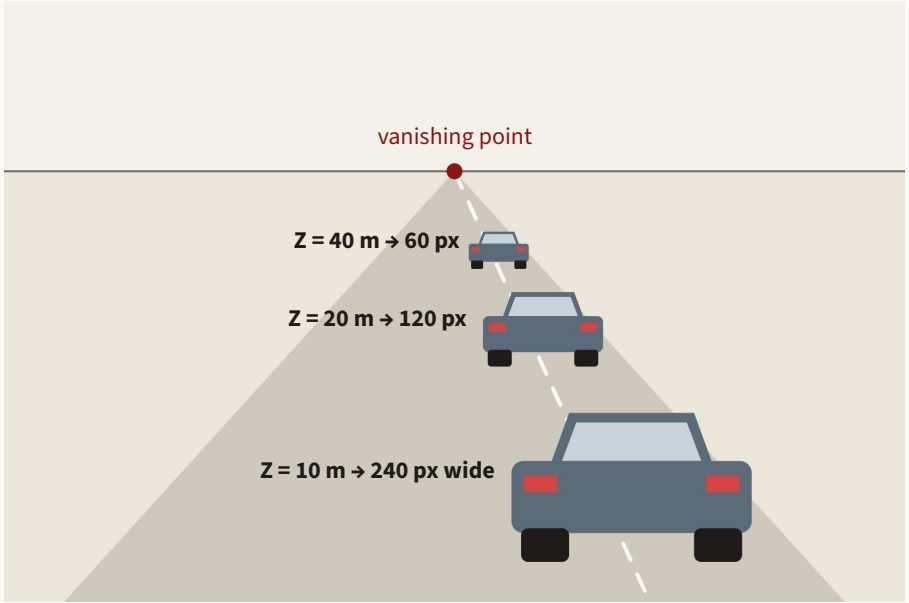


The depth effect: near is big, far is small

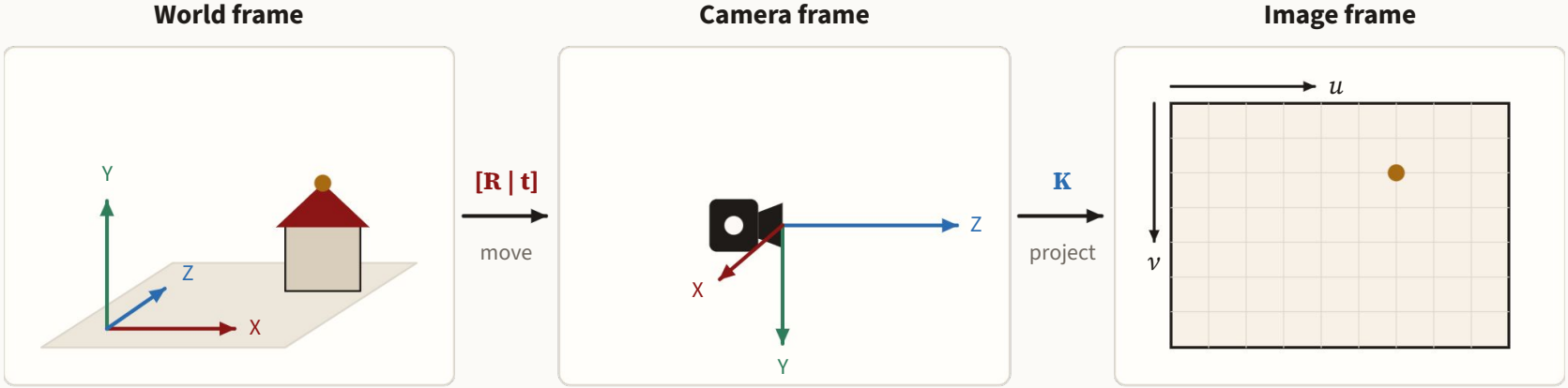
Same car. Double the distance → half the size.

$$\text{image size} = f \cdot \text{real size} / Z$$

- Parallel lane lines meet at a **vanishing point**
- This is perspective: size in the image $\propto 1 / Z$
- So a toy car up close and a real car far away can look identical



Three coordinate systems



meters · fixed to the scene

meters · moves with the camera

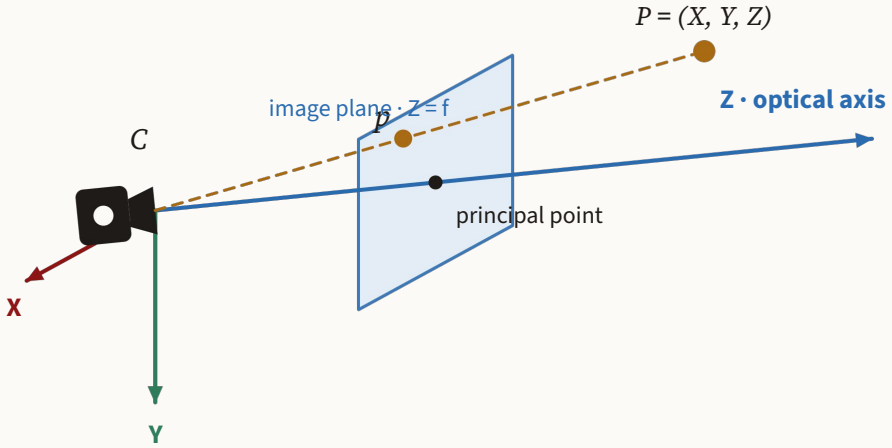
pixels · origin at top-left

A point travels: world → camera (rigid motion) → image (projection).

The camera coordinate frame

Put the origin at the camera.

- Origin C = the camera (pinhole) center
- **Z** points where the camera looks — the optical axis
- **X** right, **Y** down (the OpenCV convention)
- The image plane sits at $Z = f$
- The axis pierces it at the **principal point**



From meters to pixels

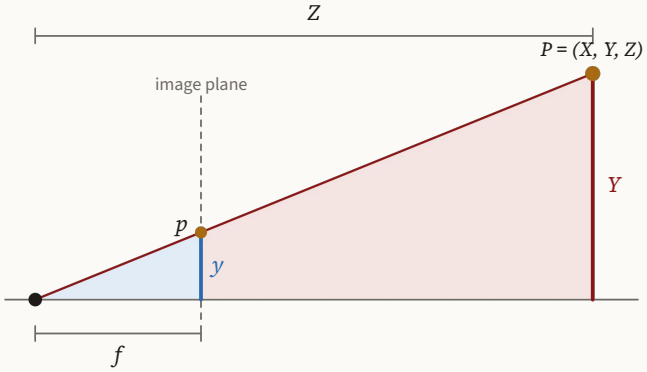
$$u = fx \cdot X / Z + cx$$
$$v = fy \cdot Y / Z + cy$$

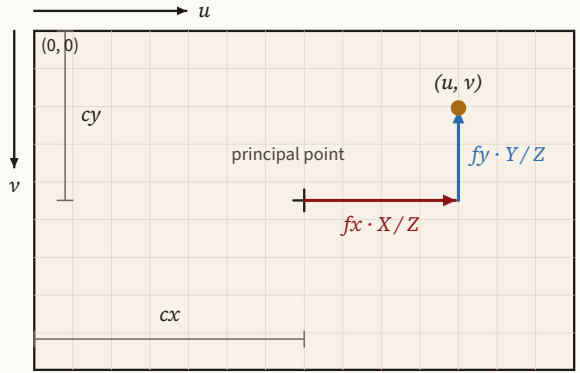
fx · X / Z

the similar-triangles result, now in **pixels** instead of meters

+ cx

move the origin from the image center to the **top-left corner**





Intrinsic parameters: the camera's personality

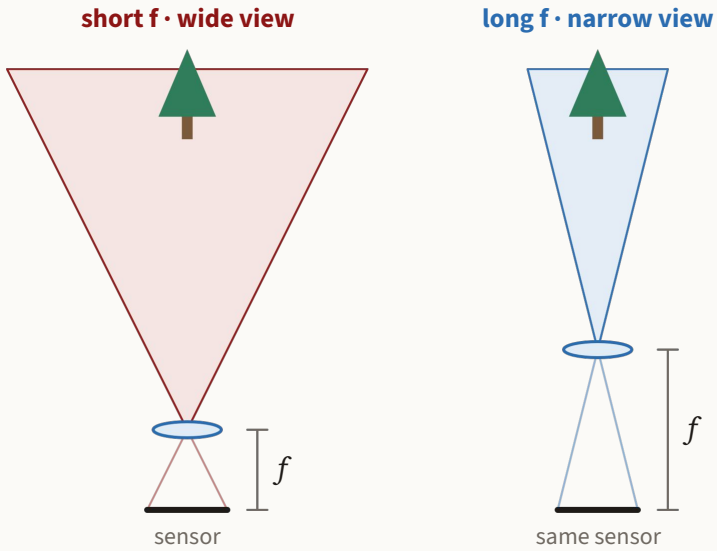
fx, fy — focal length in pixels

How strongly the lens magnifies. Longer f → narrower view, bigger objects. Two values because pixels may not be square.

cx, cy — principal point

Where the optical axis hits the sensor. Usually near, but not exactly at, the image center.

Intrinsics belong to the camera. Move the camera — they don't change.



Extrinsic parameters: where the camera is

R — rotation (3 × 3)

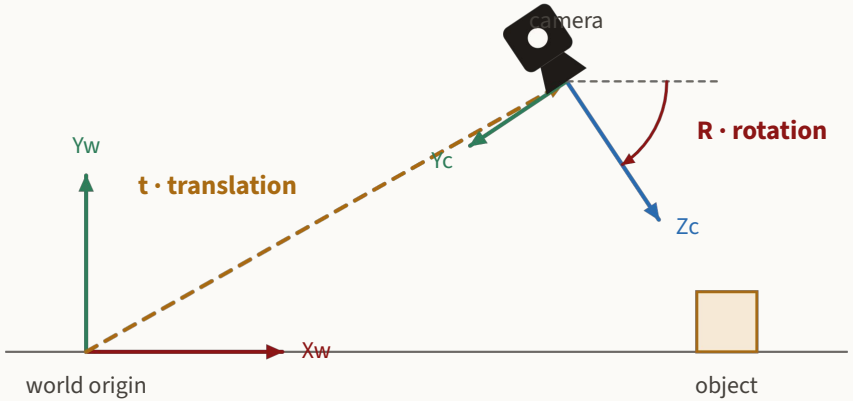
Which way the camera points. 3 numbers: roll, pitch, yaw.

t — translation (3 × 1)

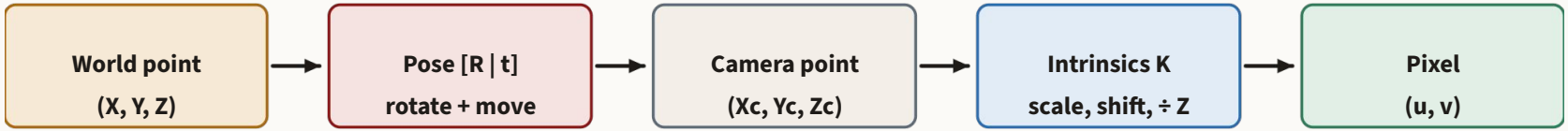
Where the camera is. 3 numbers.

$$X\text{cam} = R \cdot X\text{world} + t$$

Extrinsics change every time the camera moves.



The complete camera model



$$x \simeq K\,[\,R \mid t\,]\,X$$

"≃" means equal up to scale: divide by the third coordinate — that is the ÷ Z

$$\mathbf{P} = K\,[R \mid t] \text{ — the } 3 \times 4 \text{ camera matrix}$$

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Only now do we write matrices: they just package the steps above.

MODULE 3

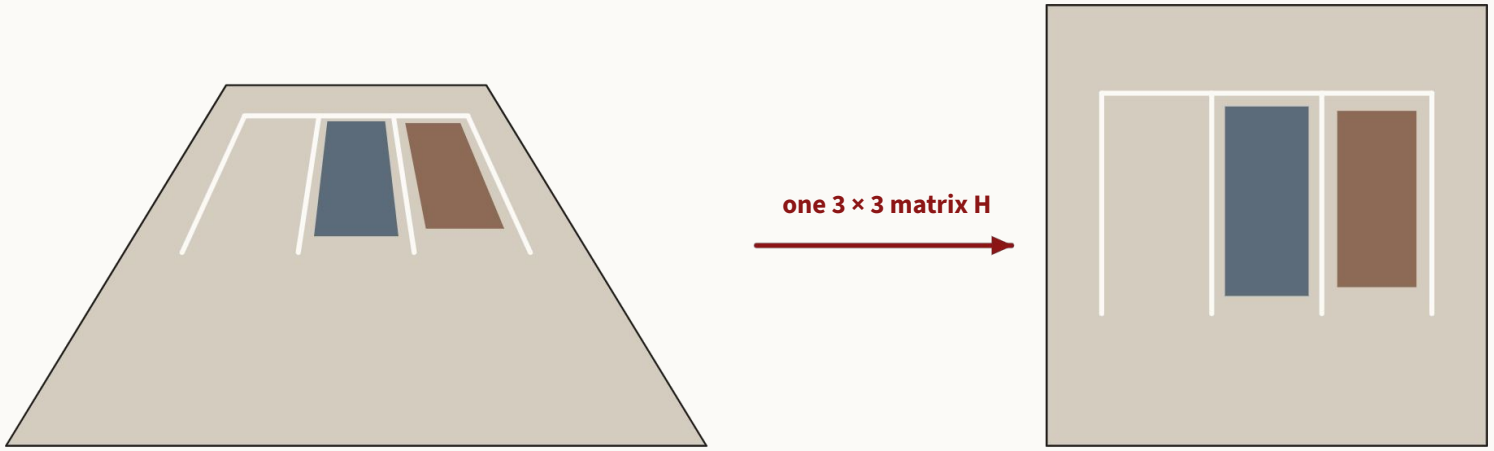
Homography

Goal — a special case: when everything lies on a plane.

03

Why 3D | Projection | Calibration | **Homography** | Stereo | Epipolar | Triangulation | Pipeline

Motivation: a bird's-eye view



Parking lines are parallel and evenly spaced in reality. One matrix undoes the perspective — because the ground is flat.

When every point lies on a plane

Full camera model for a point on the plane:

$$s\begin{bmatrix}u\\v\\1\end{bmatrix} = K\begin{bmatrix}r_1 & r_2 & r_3 & t\end{bmatrix}\begin{bmatrix}X\\Y\\Z\\1\end{bmatrix}$$

Apply the planar constraint: Z=0:

Substituting into the camera model:

$$s\begin{bmatrix}u\\v\\1\end{bmatrix} = K\begin{bmatrix}r_1 & r_2 & r_3 & t\end{bmatrix}\begin{bmatrix}X\\Y\\0\\1\end{bmatrix}$$

Expanding the matrix multiplication:

$$= K(r_1X + r_2Y + r_3 \cdot 0 + t)$$

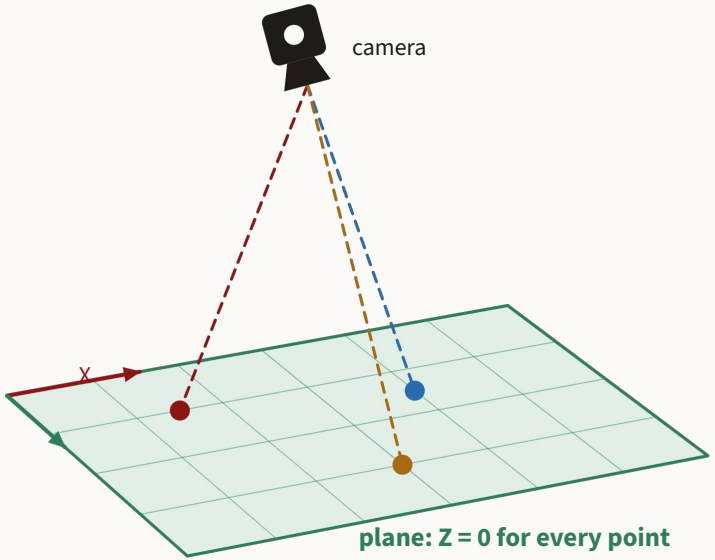
Since $r_3 \cdot 0 = 0$, the third rotation column contributes nothing.

Therefore:

$$s\begin{bmatrix}u\\v\\1\end{bmatrix} = K\begin{bmatrix}r_1 & r_2 & t\end{bmatrix}\begin{bmatrix}X\\Y\\1\end{bmatrix}$$

$$H = K\begin{bmatrix}r_1 & r_2 & t\end{bmatrix}$$

$$s\begin{bmatrix}u\\v\\1\end{bmatrix} = H\begin{bmatrix}X\\Y\\1\end{bmatrix}$$



What is a homography?

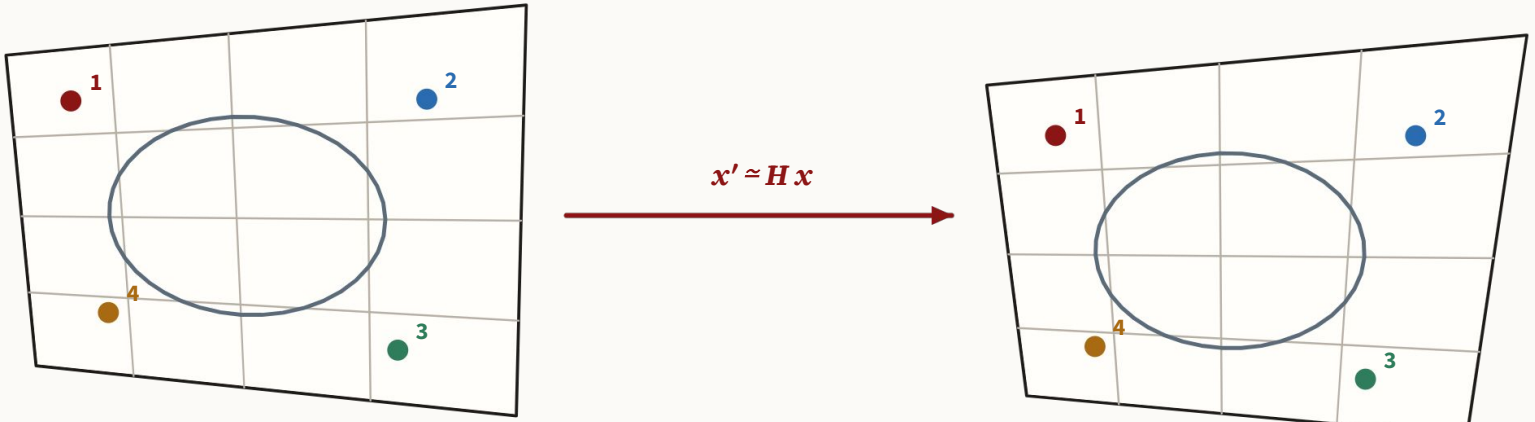


Image 1

Image 2

Two views of the same plane are related by one 3 × 3 matrix **H**.

9 entries, but only defined up to scale → **8 degrees of freedom**

Seeing it: document scanning

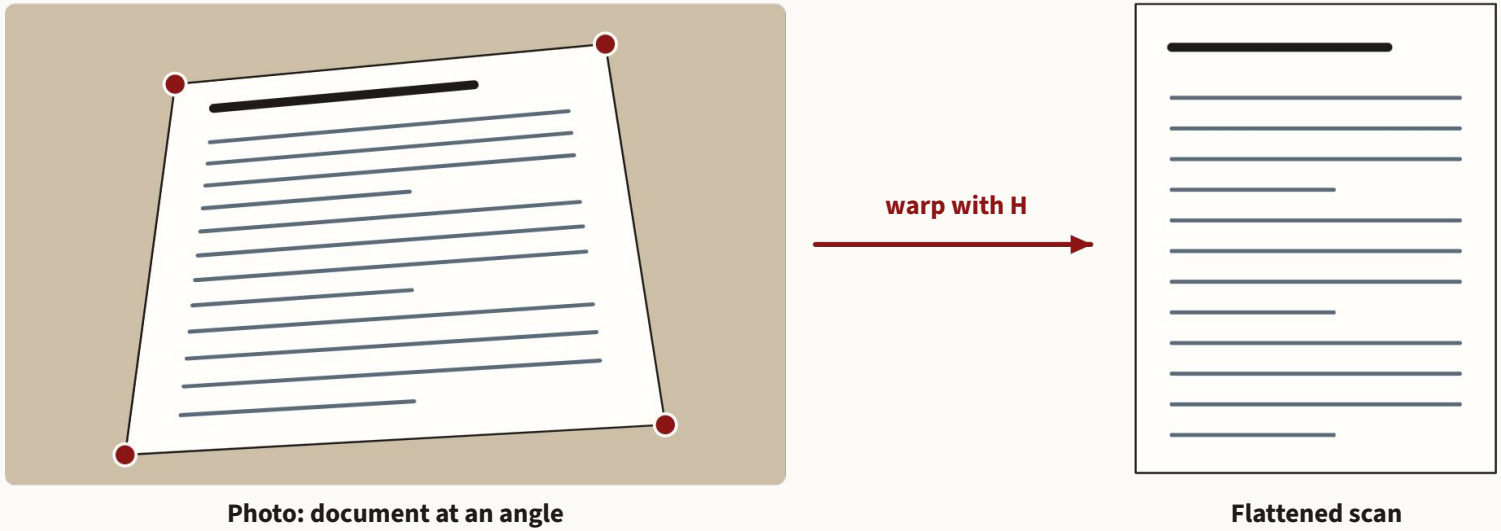


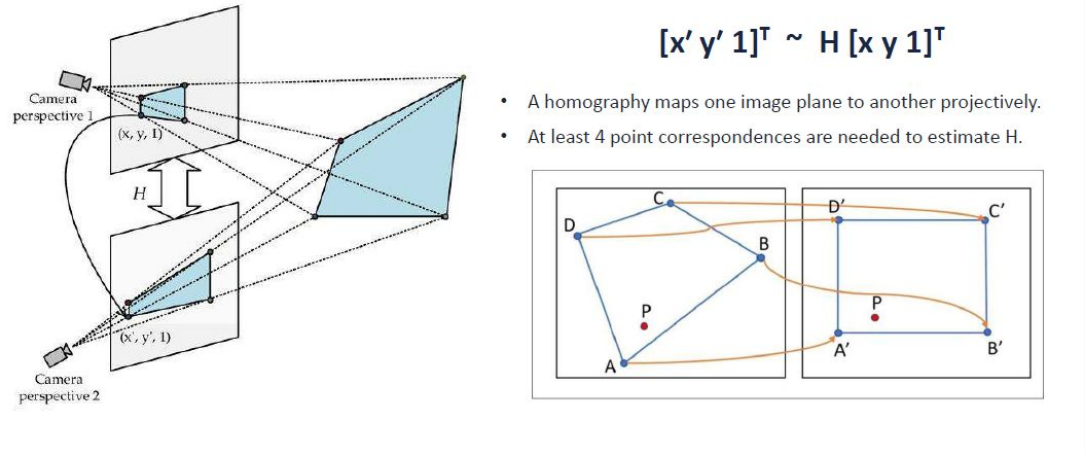
Photo: document at an angle

Flattened scan

Tap the 4 corners → compute **H** → resample every pixel.

```
cv2.warpPerspective(img, H, (w, h))
```

Why exactly 4 points?



	Count
Unknowns in H	9 − 1 (scale) = 8
Equations per point pair	2 (one for u, one for v)
Point pairs needed	8 ÷ 2 = 4

Condition: no three of the four points on one line.

In practice

Use many matches and **RANSAC** to ignore bad ones:

cv2.findHomography(src, dst, cv2.RANSAC)

Why exactly 4 points?

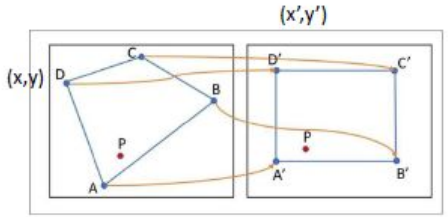
$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

$$x' \sim Hx$$

The ~ means "equal up to scale.", this is important as now we have

$$9 - 1 = \boxed{8}$$

Independent Parameters



The homography says:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

If you expand this, you get two independent equations:

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$

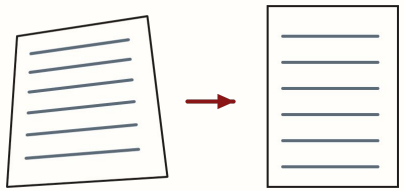
$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

So:

> **ONE point correspondence gives us 2 equations.**

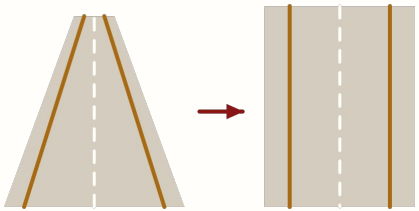
Therefore to solve for 8 independent variables we at least need 4 points of correspondence

Applications



Document scanning

Tilted photo → flat page



Inverse perspective mapping

Converging lane lines → parallel, measurable lanes

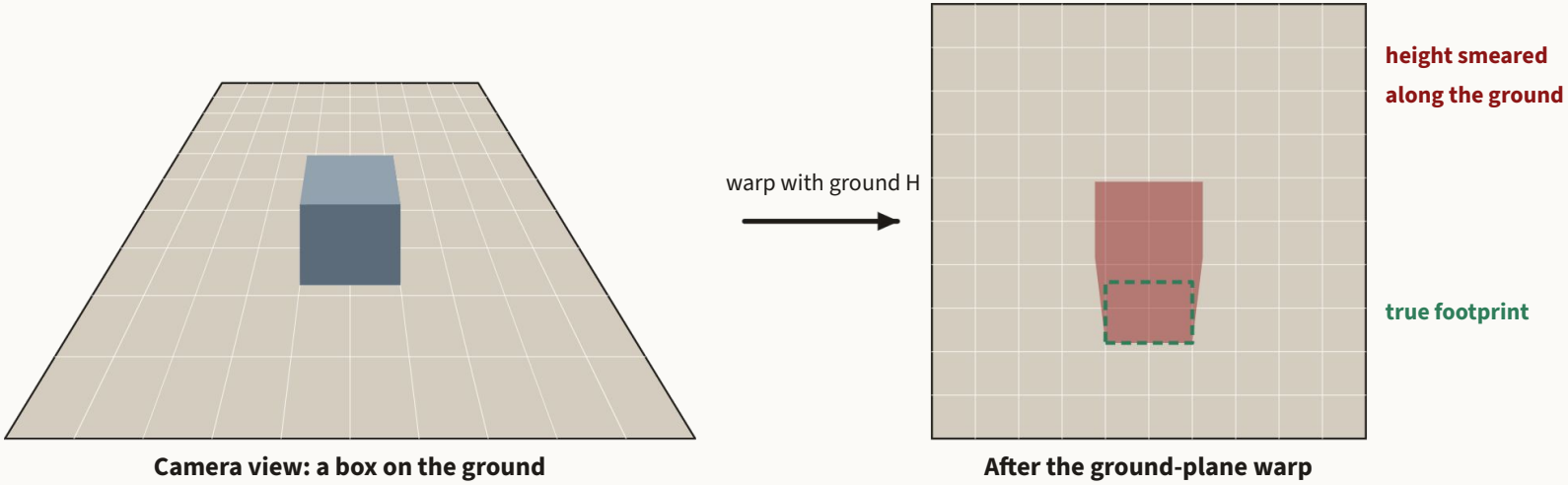


Surround-view BEV

Four fisheye cameras warped to the ground and stitched around the car

All three assume the same thing: **the content lies on one plane**.

Limitations: the world is not flat



H is exact only for points **on the plane** (or a purely rotating camera). Anything with height gets smeared.

MODULE 2

Camera Calibration

02

A camera takes a 3D point and projects it onto a 2D image.

We want to estimate the parameters that explain this projection

(find K, R and t)

Why calibrate? A wrong focal length

A car is **1.5 m** tall and appears **60 px** tall in the image. How far away is it?

$$Z = f \cdot H / h$$

25 m

with the calibrated focal length **f = 1000 px**

20 m

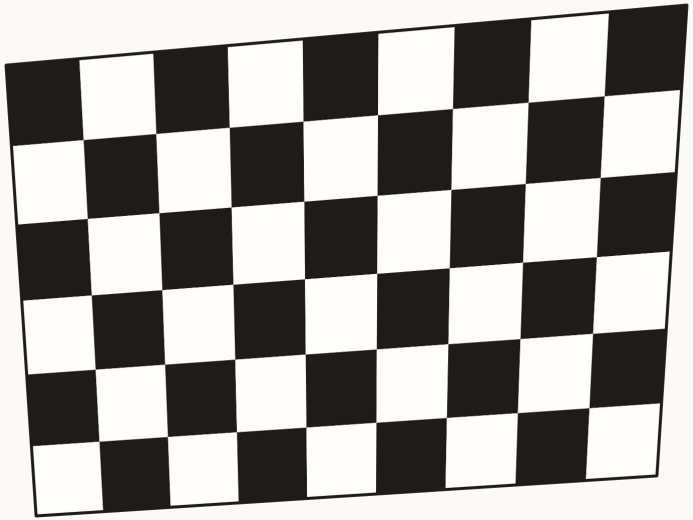
with a guessed focal length **f = 800 px**

0.36 s

the 5 m error, at 50 km/h — the difference between braking in time and not

Every 3D measurement inherits the calibration error.

Problem Definition



Need estimation of K, R, T. from know 3d points relative to each other and their 2d corresponding points:

$$x \simeq K \left[R \mid t \right] X$$

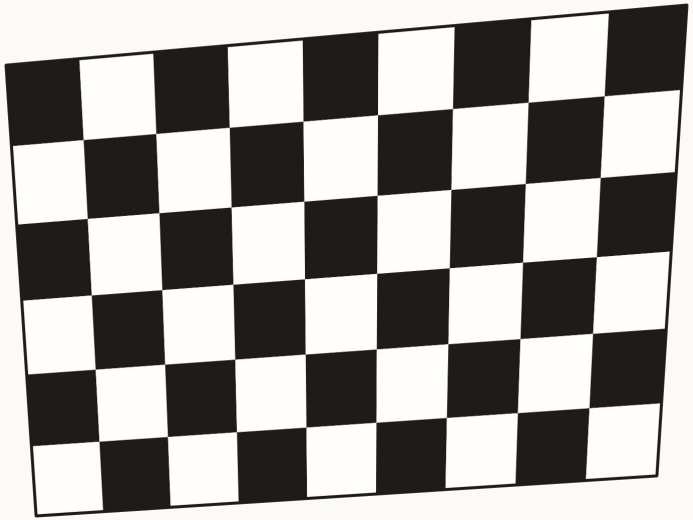
“≃” means equal up to scale: divide by the third coordinate — that is the ÷ Z

4 intrinsic unknowns: fx, fy, cx, cy

6 extrinsics unknowns:

- R: Rotation matrix — 3 degrees of freedom.
- t : Translation vector — 3 degrees of freedom

Why everyone uses a checkerboard



Known geometry

Measure the square size once (e.g. 25 mm) and every corner position is known.

Precise corners

Black–white X-junctions can be located to a fraction of a pixel.

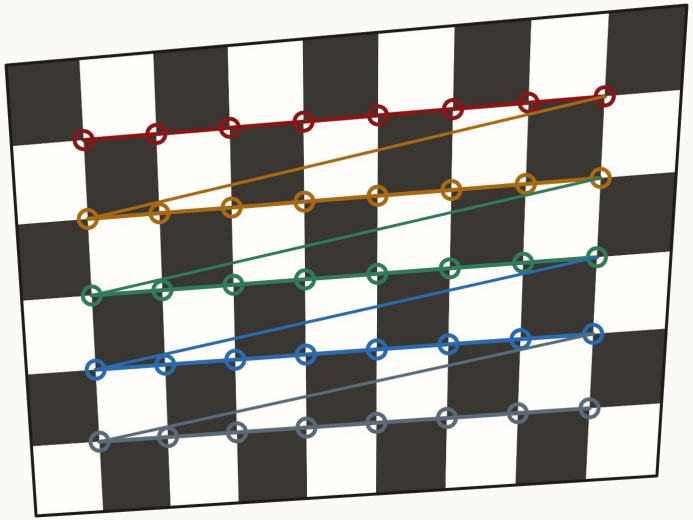
Easy to find automatically

High contrast, regular pattern, cheap to print.

Flat

All corners share $Z = 0$ — this simplifies the math (see homography).

Detecting the corners



Detect: Find the checkerboard grid and its corners.

Order: Arrange corners consistently to match known 3D points.

Refine: Improve corner locations to sub-pixel accuracy.

```
cv2.findChessboardCorners(img, (8, 5))
cv2.cornerSubPix(gray, corners, ...)
```

Output per image: 40 pixel positions, in a known order.

corners detected in the image might be:

(152.4,210.7), (174.8,211.2),…

Known 3D coordinates

Put the world origin on the board.

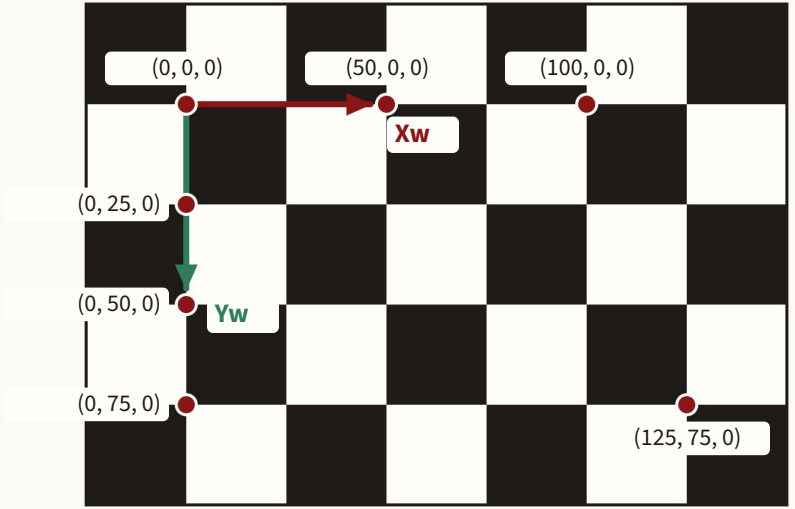
- Every corner then has $Z = 0$
- Corner (i, j) sits at $(i \cdot s, j \cdot s, 0)$
- Each photo gives ~40 pairs: **known 3D point ↔ detected pixel**
- Those pairs are the data calibration fits

We define its 3D coordinates on a flat plane:

Z=0

For example:

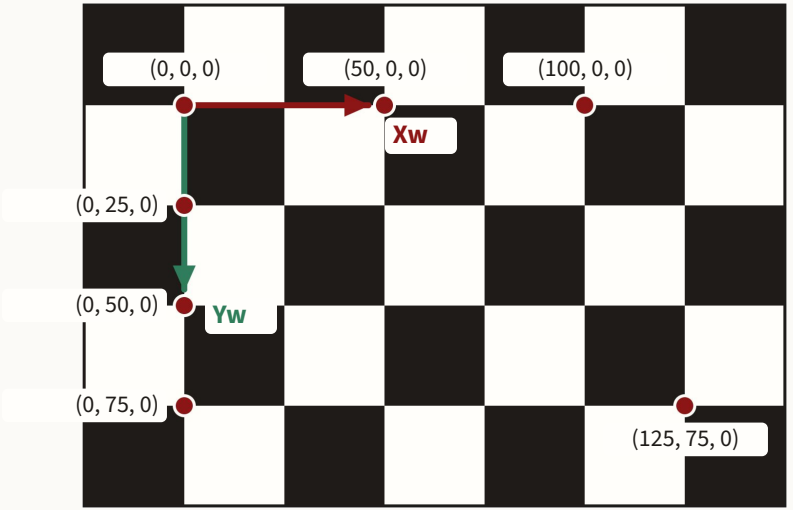
(0,0,0), (25,0,0), (50,0,0),…



units: squares — multiply by the square size s

Known 3D coordinates

$$H = K \begin{bmatrix} r_1 & r_2 & t \end{bmatrix}$$

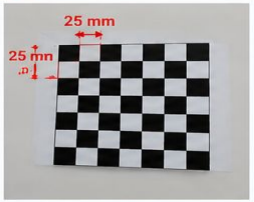


units: squares — multiply by the square size s

Camera Calibration using a Checkerboard (Example: 25 mm squares)

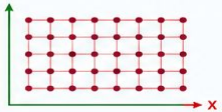
Estimate camera intrinsics **K**, **distortion**, and **per-image pose (R, t)** from multiple images of a known checkerboard

1 Checkerboard and 3D points



Checkerboard with 25 mm squares

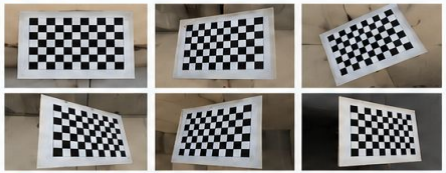
9 × 6 inner corners
(10 × 7 squares)



3D coordinates (Z = 0)

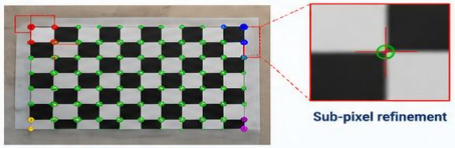
$$X_{i,j} = (i \cdot 25,\ j \cdot 25,\ 0)\ \text{mm}$$
$$i = 0 \ldots 8,\quad j = 0 \ldots 5$$

2 Capture multiple images



- Capture 10–20 images
- Vary position, orientation and distance
- Cover different parts of the image
- Ensure the board is fully visible

3 Detect and refine corners



Detected inner corners

Sub-pixel refinement

- Find the checkerboard grid (threshold + contours)
- Order corners consistently (row by row)
- Refine corner locations to sub-pixel accuracy

4 2D–3D correspondences

Index (*i*, *j*)	3D point (*X*, *Y*, *Z*) [mm]	Detected image point (*u*, *v*) [pixels]
(0, 0)	(0, 0, 0)	(152.4, 210.7)
(1, 0)	(25, 0, 0)	(174.8, 211.2)
(2, 0)	(50, 0, 0)	(197.1, 211.6)
(0, 1)	(0, 25, 0)	(151.9, 233.0)
(1, 1)	(25, 25, 0)	(174.2, 233.6)
…	…	…

5 Planar homography for each image

All points lie on a plane ($Z = 0$), so each image gives a homography $\boldsymbol{H}$ between plane coordinates (X, Y) and image points (u, v):

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \boldsymbol{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}, \qquad \boldsymbol{H} = K \begin{bmatrix} \boldsymbol{r}_1 & \boldsymbol{r}_2 & \boldsymbol{t} \end{bmatrix}$$

- $\boldsymbol{H}$ is a 3 × 3 matrix (8 DoF, defined up to scale).
- Estimated from 4+ non-collinear point correspondences (we use all detected corners).

6 Estimate camera intrinsics K

From homographies of multiple images, solve for the intrinsic matrix $\boldsymbol{K}$. $\boldsymbol{K}$ has 5 unknowns (skew usually 0):

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Intrinsic parameters (unknowns)

- f_x: focal length (pixels)
- f_y: focal length (pixels)
- c_x: principal point (pixels)
- c_y: principal point (pixels)
- γ: skew (usually 0)

- Need at least 3 different views (provides constraints).
- Use 10–20 images for robust estimation.

7 Estimate R and t for each image

Once $\boldsymbol{K}$ is known, recover rotation and translation for each image using its homography $\boldsymbol{H}$:

$$\bar{\boldsymbol{r}}_1 = K^{-1}\mathbf{h}_1,\quad \bar{\boldsymbol{r}}_2 = K^{-1}\mathbf{h}_2,\quad \bar{\boldsymbol{t}} = K^{-1}\mathbf{h}_3$$
$$\boldsymbol{r}_1 = \lambda\bar{\boldsymbol{r}}_1,\quad \boldsymbol{r}_2 = \lambda\bar{\boldsymbol{r}}_2,\quad \boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2,\quad \boldsymbol{t} = \lambda\bar{\boldsymbol{t}}$$

- $R = [\,\boldsymbol{r}_1\ \ \boldsymbol{r}_2\ \ \boldsymbol{r}_3\,]$ is a valid rotation matrix (3 DoF)
- $\boldsymbol{t}$ is the translation vector (3 DoF)

8 Estimate lens distortion

Model radial and tangential distortion, e.g.:

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2\,p_1 xy + p_2(r^2 + 2x^2)$$
$$y_d = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy$$

- Estimate distortion coefficients (e.g., k_1, k_2, k_3, p_1, p_2).
- Refine $\boldsymbol{K}$, $\boldsymbol{R}$, $\boldsymbol{t}$ and distortion jointly by minimizing reprojection error (non-linear least squares / Levenberg–Marquardt).

9 Final result and validation

Outputs

- Intrinsic matrix **K** (e.g., f_x. f_y, c_x, c_y)
- Distortion coefficients (e.g., k_1, k_2, p_1, p_2)
- Rotation **R** and translation **t** for each image
- Reprojection error (in pixels)

Typical reprojection error

~ 0.1 – 0.5 pixels (good calibration)

Reprojection example



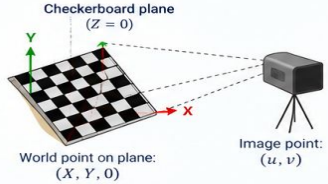
Detected corner

Reprojected corner

Estimating Camera Intrinsics K from Homographies (Zhang's Method)

Use rotation constraints to eliminate R, t and solve for K from multiple checkerboard views

1 Planar checkerboard projection



For a planar pattern ($Z = 0$):

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_1 & r_2 & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

$$H = K \begin{bmatrix} r_1 & r_2 & t \end{bmatrix}$$

H is the homography from checkerboard plane to image.

2 Homography columns

Let $H = \begin{bmatrix} h_1 & h_2 & h_3 \end{bmatrix}$, where

$$h_1 = \lambda K r_1$$
$$h_2 = \lambda K r_2$$
$$h_3 = \lambda t$$

- λ is an unknown scale fector (homography is up to scale).
- We use only h_1 and h_2 to estimate K (translation t is not needed).

3 Properties of rotation matrix

$R = \begin{bmatrix} r_1 & r_2 & r_3 \end{bmatrix}$ is a valid rotation matrix, so its columns are orthonormal unit vectors:

$$r_1^T r_2 = 0 \quad \text{(perpendicular)}$$
$$r_1^T r_1 = r_2^T r_2 = 1 \quad \text{(equal length)}$$

These properties hold for ***any*** rotation matrix (independent of the checkerboard). We use them to eliminate R from the equations.

4 Eliminate R using the constraints

From $h_1 = \lambda K r_1$ and $h_2 = \lambda K r_2$, we have

$$r_1 = \frac{1}{\lambda} K^{-1} h_1 \qquad r_2 = \frac{1}{\lambda} K^{-1} h_2$$

Apply rotation constraints:

1. Orthogonality

$$r_1^T r_2 = 0$$
$$\left(\frac{1}{\lambda} K^{-1} h_1\right)^T \left(\frac{1}{\lambda} K^{-1} h_2\right) = 0$$
$$h_1^T K^{-T} K^{-1} h_2 = 0$$
$$h_1^T B h_2 = 0$$

2. Equal length

$$r_1^T r_1 = r_2^T r_2$$
$$h_1^T K^{-T} K^{-1} h_1 = h_2^T K^{-T} K^{-1} h_2$$
$$h_1^T B h_1 - h_2^T B h_2 = 0$$

where $B = K^{-T} K^{-1}$ (symmertic matrix, 6 unknowns up to scale).

5 Linear equations for B

Let $B = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{12} & B_{22} & B_{23} \\ B_{13} & B_{23} & B_{33} \end{bmatrix}$ and

$$b = \begin{bmatrix} B_{11} & B_{12} & B_{13} & B_{22} & B_{23} & B_{33} \end{bmatrix}^T.$$

For a homography column $h_i = [h_{1i}, h_{2i}, h_{3i}]^T$, define

$$v_{ij} = \begin{bmatrix} h_{1i}h_{1j} \\ h_{1i}h_{2j} + h_{2i}h_{1j} \\ h_{1i}h_{3j} + h_{3i}h_{1j} \\ h_{2i}h_{2j} \\ h_{2i}h_{3j} + h_{3i}h_{2j} \\ h_{3i}h_{3j} \end{bmatrix}$$

Then $h_i^T B h_j = v_{ij}^T b$.

For each image, we get two linear equations:

$$v_{12}^T b = 0 \qquad (v_{11} - v_{22})^T b = 0$$

6 Solve for B using multiple views

- Each checkerboard image provides 2 equations.
- Stack equations from N images:

$$V b = 0 \qquad V \in \mathbb{R}^{2N \times 6}$$

- Need at least 3 diverse views (6 equations) for the minimal solution (since b has 6 unknowns up to scale).
- In practice, use 10–20 images for robustness.
- Solve using SVD: the solution b is the right singular vector corresponding to the smallest singular value.

7 Recover K from B

We have $B = K^{-T} K^{-1}$. Compute $A = B^{-1} = K K^T$ and recover K using Cholesky decomposition (upper triangular matrix):

$$A = K K^T \quad \Rightarrow \quad K = \text{chol}(A) \text{ (upper triangular)}$$

This gives the intrinsic matrix:

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Intrinsic parameters (5 unknowns)

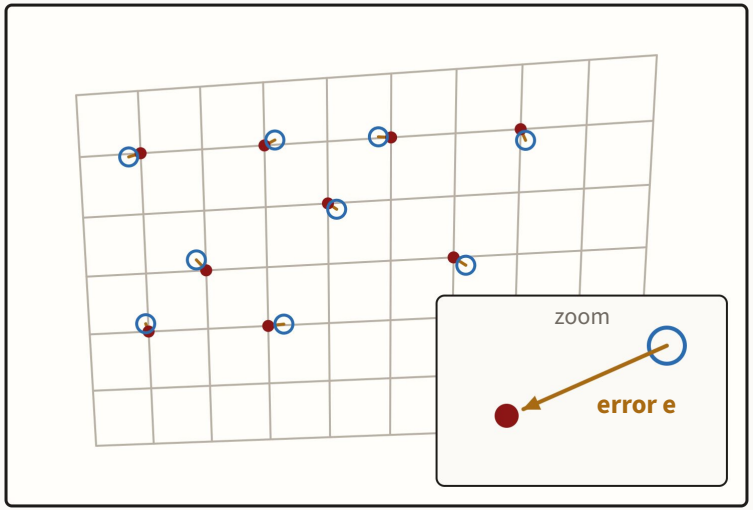
- f_x, f_y : focal lengths (pixels)
- c_x, c_y : principal point (pixels)
- γ : skew (usually 0)

8 Summary pipeline

1. Detect corners and estimate H_i for each image
2. For each H_i, form v_{12} and $(v_{11} - v_{22})$
3. Stack all equations $Vb = 0$ and solve for B using SVD
4. Recover K from $B = K^{-T} K^{-1}$
5. Compute R, t for each image (optional)

Reprojection error — the key idea

● observed (detected corner) ○ predicted (current K, R, t)



1. Take a known 3D corner X
2. Project it with the current K, R, t → predicted pixel $\hat{x}$
3. Compare with the detected pixel x

$$E = \Sigma \parallel x - \hat{x} \parallel^2$$

minimize over K, distortion, and every view's R, t

Rule of thumb

RMS error well under ~0.5 px is good. Much larger → blurry photos, wrong corners, or a bad distortion model.

MODULE 5

Epipolar Geometry

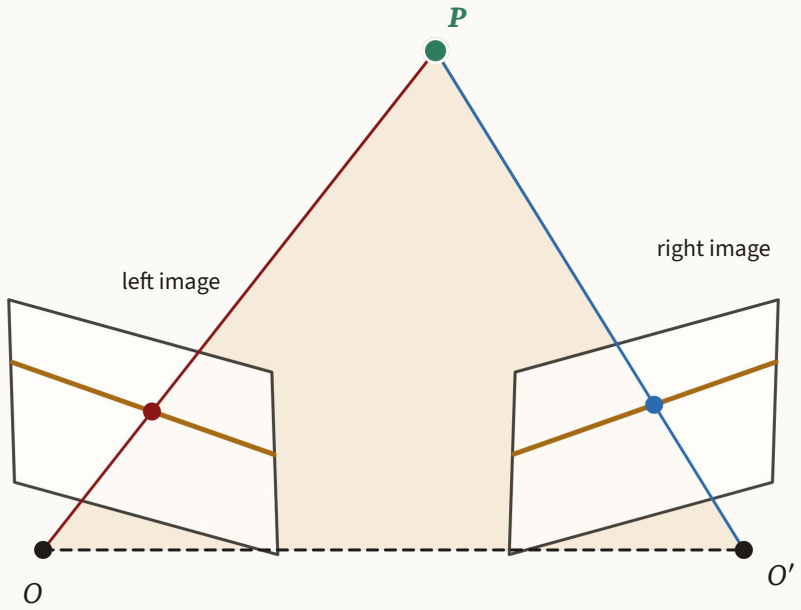
05

Epipolar geometry tells us where a corresponding point can appear in the other camera's image.

Without Epipolar Geometry



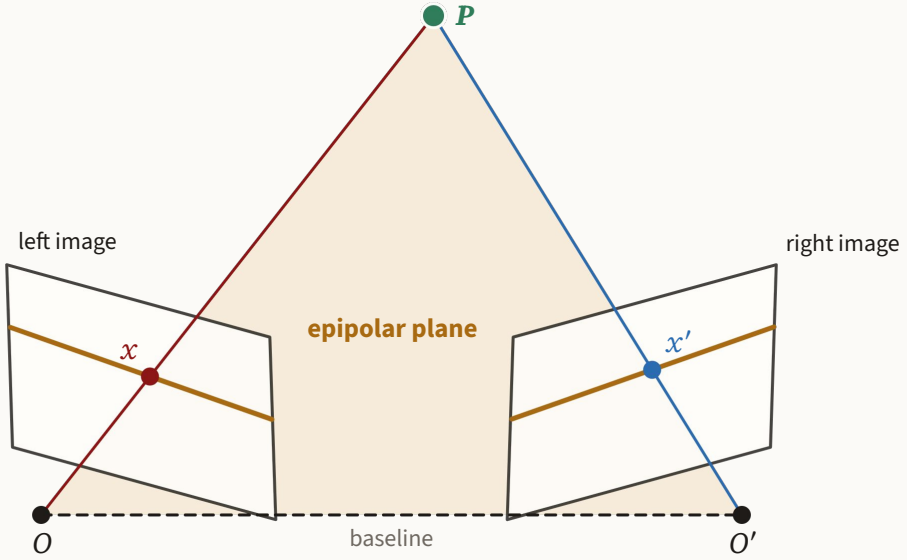
If we know the camera origin



The epipolar plane

Three points define a plane.

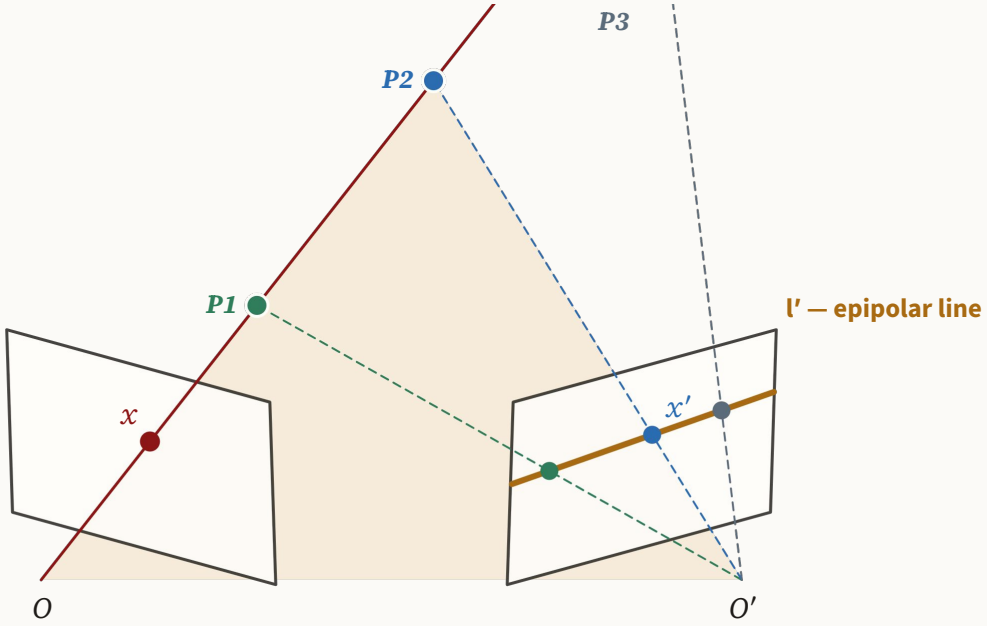
- Camera centers O, O' and the 3D point P
- The plane through them is the **epipolar plane**
- It slices each image along a line called **epipolar lines**
- Both x and x' must lie in this plane



The epipolar line

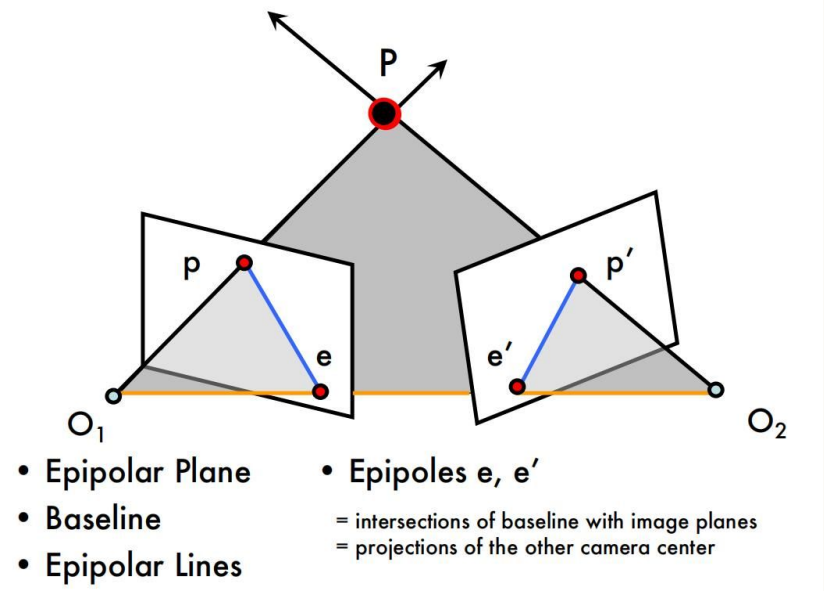
The match of x must lie on the line l'.

- We don't know how far along its ray P is: P_1, P_2 or P_3…
- Each candidate projects somewhere in the right image
- All of those projections fall on **one line**, l'
- So we search along l', not the whole image



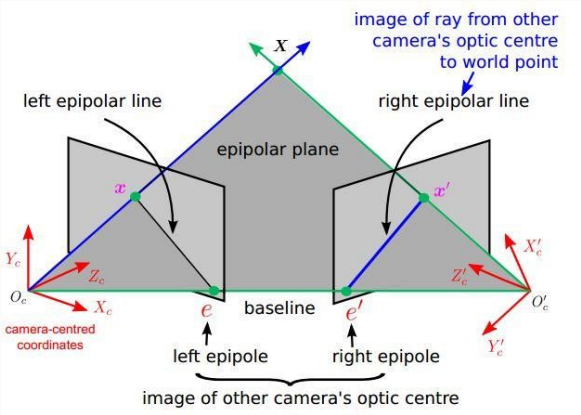
The epipole

The **epipole** *e′* is where the other camera's center appears in this image. Every epipolar line passes through it.

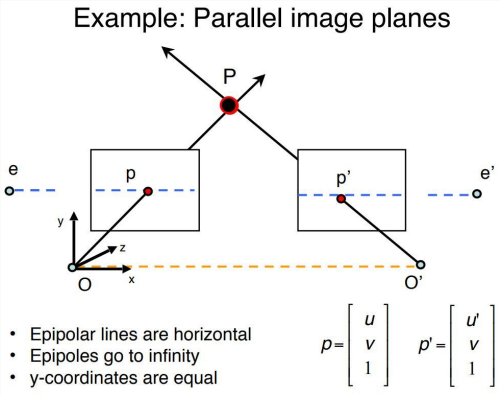


The epipole

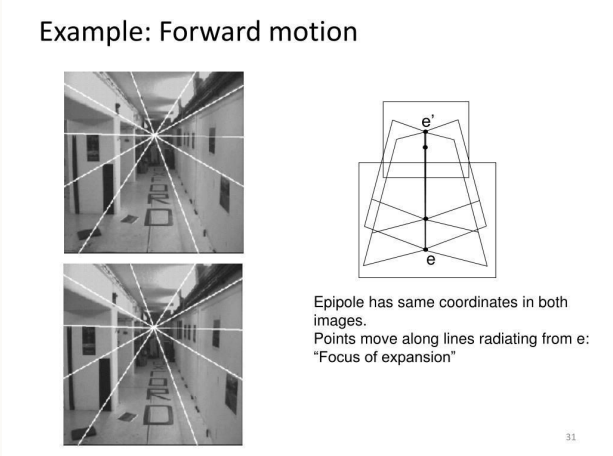
The **epipole** *e′* is where the other camera's center appears in this image. Every epipolar line passes through it.



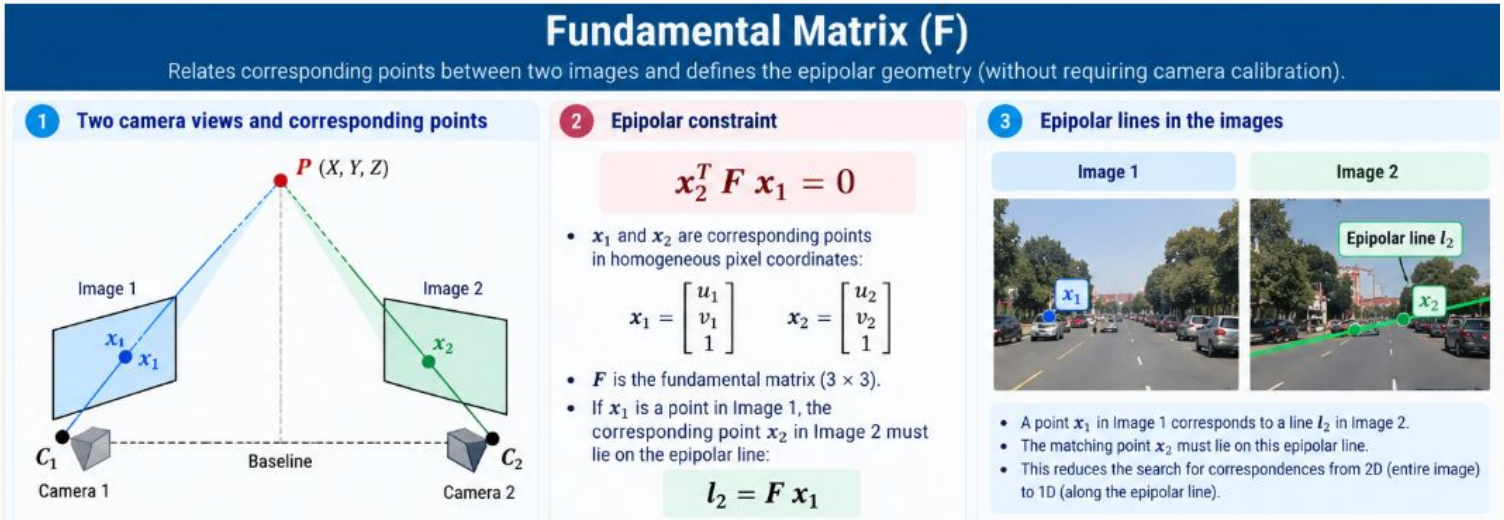
General: lines meet at e′
(often outside the image)



lines are parallel rows
Rectified stereo: e′ at infinity



Driving forward: e′ at the center
"focus of expansion"



Works with **uncalibrated** cameras. Estimate it from ≥ 8 matches (the 8-point algorithm).

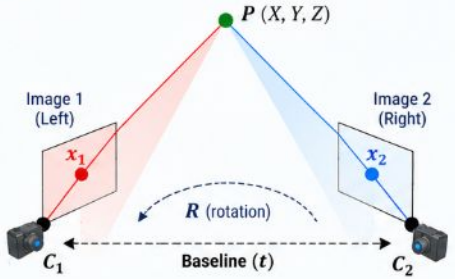
```
cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC)
```

The essential matrix: when cameras are calibrated

Essential Matrix (E)

Relates corresponding points between two calibrated camera views and directly encodes the relative rotation and translation.

1 Geometry: Two calibrated cameras



2 Essential matrix equation

$$\hat{x}_2^T E \hat{x}_1 = 0$$

- $\hat{x}_1$ and $\hat{x}_2$ are corresponding points in normalized camera coordinates (intrinsics removed).
- E is the essential matrix (3 × 3).
- If $\hat{x}_1$ is a point in Image 1, the matching point $\hat{x}_2$ in Image 2 must lie on the epipolar line given by:

$$l_2 = E\hat{x}_1$$

3 Relation to camera motion

$$E = [t]_\times R$$

- R : Relative rotation from camera 1 to camera 2 (3 × 3).
- t : Relative translation direction (3 × 1, up to scale).
- $[t]_\times$: Skew-symmetric matrix of t.

$$[t]_\times = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

E directly encodes the relative rotation and translation (up to scale) between the two calibrated cameras.

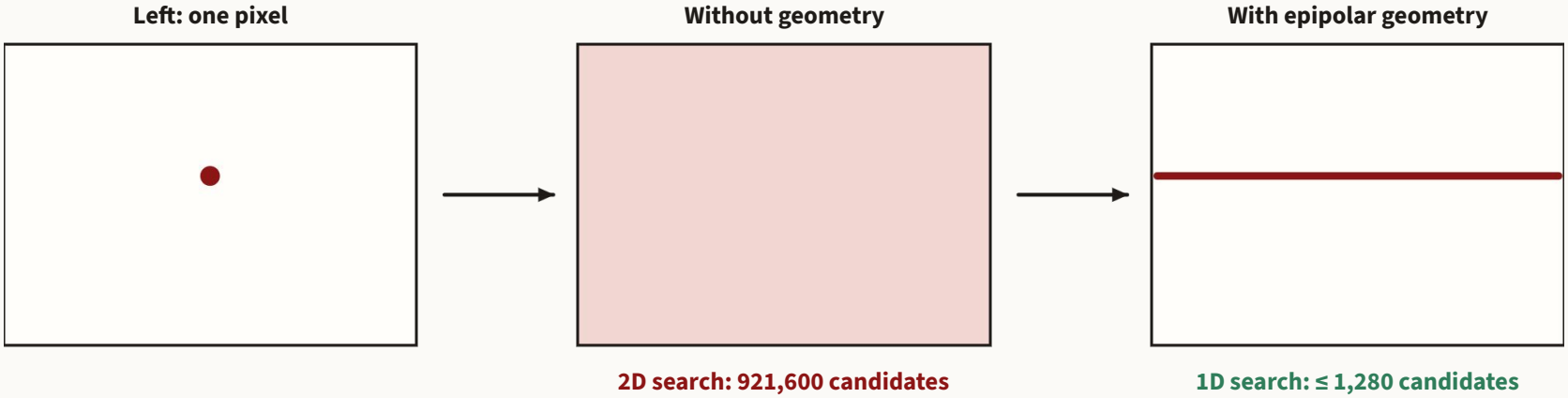
Calibrate first → E → recover how the second camera moved.

The fundamental vs essential matrix

	Fundamental F	Essential E
Works on	Raw pixels	Normalized coords, $K^{-1}x$
Needs K?	No	Yes (calibrated)
Degrees of freedom	7	5
Gives you	Epipolar lines	Relative pose: R and t (t up to scale)

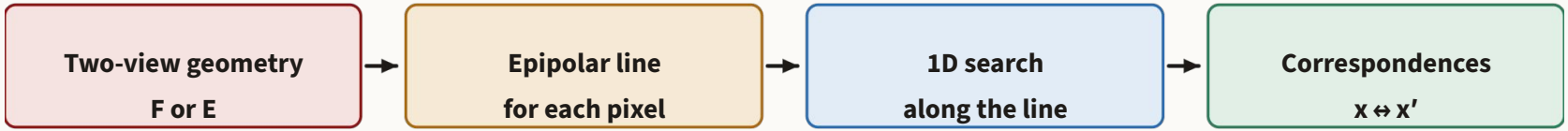
The Fundamental Matrix only tells you how pixels correspond on a screen, but the Essential Matrix tells you how the physical camera moved in the 3D world.

Search reduction: whole image → one line



720× fewer candidates per pixel — faster, and far fewer look-alike mistakes.

Summary: geometry makes matching tractable



Geometry says where to look
The epipolar constraint restricts the match to a line.

Appearance says what matches
Compare patches (or learned features) along that line.

Rectified special case
Lines become rows, so the match is just a horizontal shift: disparity.

Epipolar geometry → correspondence. Correspondence → depth.

MODULE 6

Triangulation

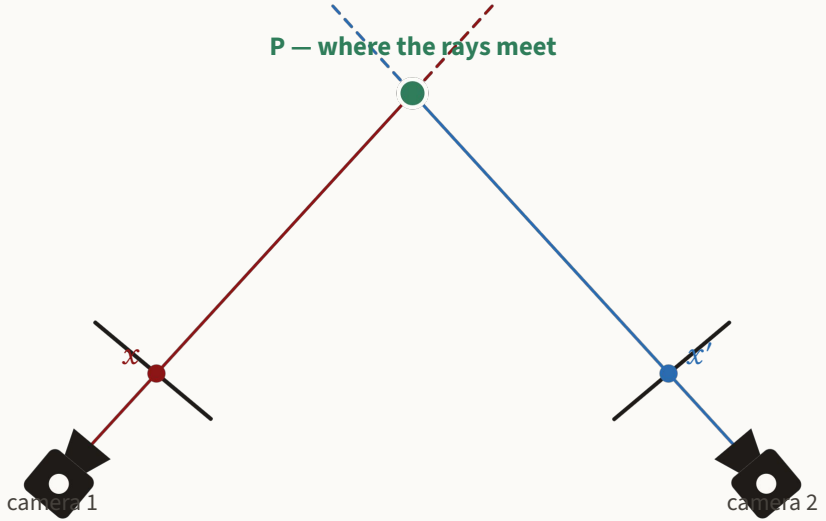
Goal — recover the 3D point from two matched pixels.

06

Two rays from two cameras

Each pixel defines a **ray**.

- The ray holds every 3D point that could have made that pixel
- Two cameras → two rays
- Where they cross is the 3D point *P*
- We need: both cameras' *K, R, t* (calibration) and the match $x \leftrightarrow x'$ (epipolar search)



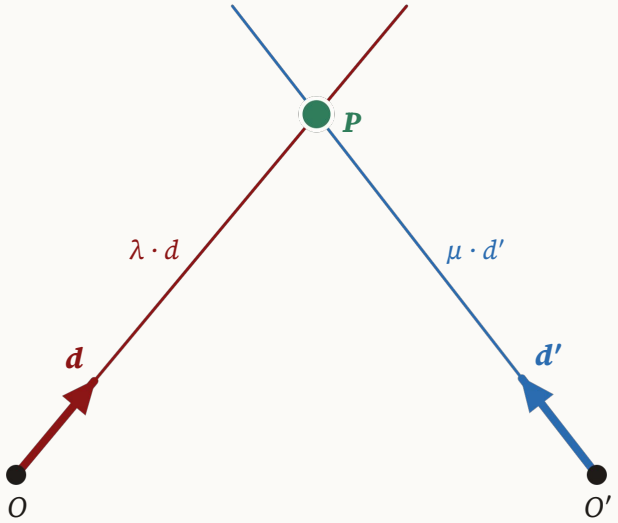
The ideal case

Perfect pixels, perfect calibration → the rays meet exactly.

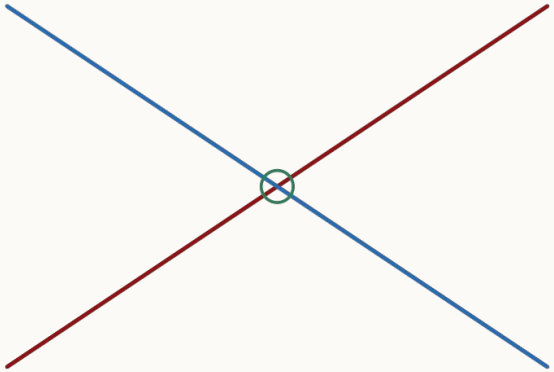
$$O + \lambda\, d = O' + \mu\, d'$$

Solve for the two distances λ and μ, then $P = O + \lambda d$.

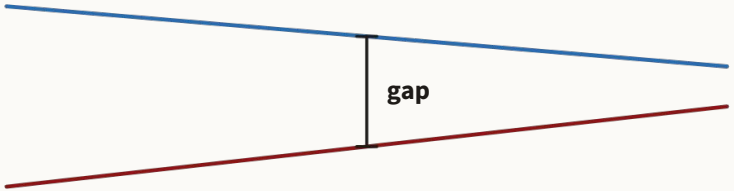
Three equations (x, y, z), two unknowns → normally no exact solution. Which brings us to reality…



The real case: rays don't meet



Top view: they seem to cross



Side view: they pass each other

Pixel noise
Corners are found to ± a fraction of a pixel

Calibration error
K, R, t are estimates, not truth

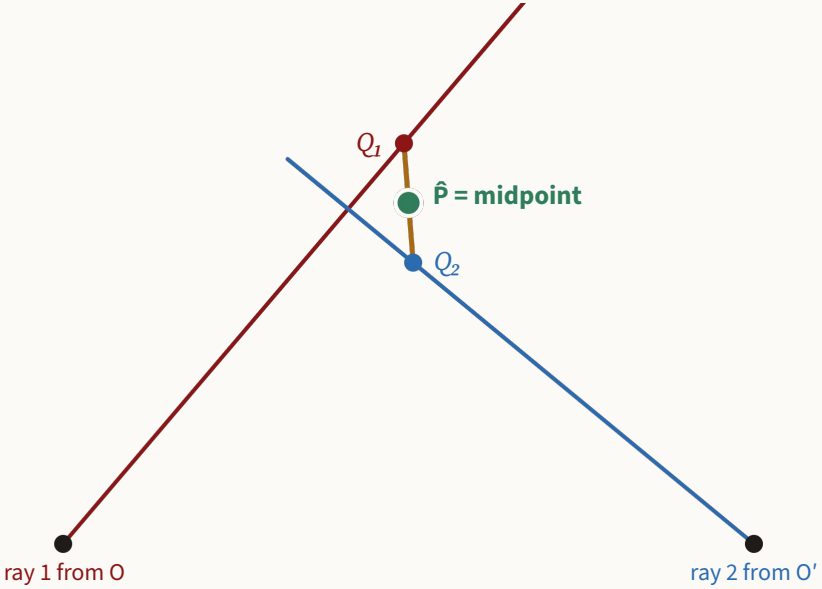
Quantization
Pixels are discrete

Triangulation: the closest point

Take the closest point between the two rays.

1. Find the shortest segment Q_1Q_2 joining the rays — it is perpendicular to both
2. Its midpoint $\hat{P}$ is the estimate

> **Better, and standard**
> Choose $\hat{P}$ whose reprojections land closest to the observed pixels — minimize the **reprojection error**, as in calibration.



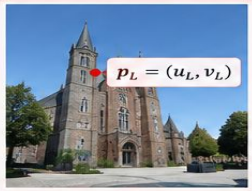
Triangulation: Recovering a 3D Point from Two Views

Given corresponding points in two images and known camera geometry, triangulation estimates the 3D position of the point.

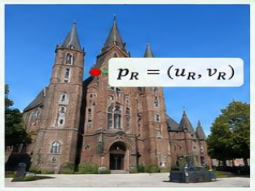
1 Find corresponding points

Identify the same physical point in both images using feature matching.

Left image (I_L)



Right image (I_R)

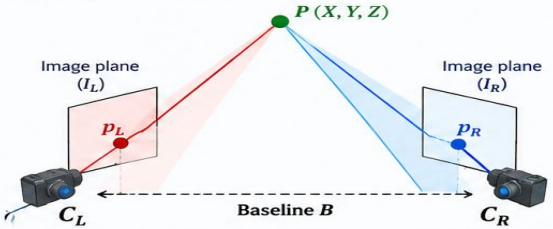


Corresponding image points:

$$p_L = (u_L, v_L)$$
$$p_R = (u_R, v_R)$$

2 Back-project into 3D rays

Using camera calibration, each pixel corresponds to a ray in 3D originating from its camera center.



Convert pixel to normalized camera coordinates:

$$\hat{x}_L = K_L^{-1} \begin{bmatrix} u_L \\ v_L \\ 1 \end{bmatrix}$$

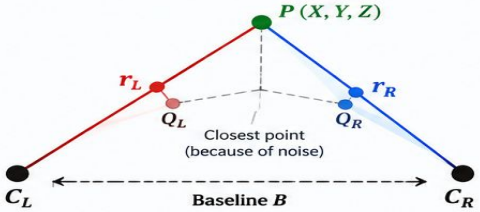
$$\hat{x}_R = K_R^{-1} \begin{bmatrix} u_R \\ v_R \\ 1 \end{bmatrix}$$

Each defines a 3D ray:

$$r_L = C_L + \lambda_L \hat{x}_L$$
$$r_R = C_R + \lambda_R \hat{x}_R$$

3 Intersect the rays

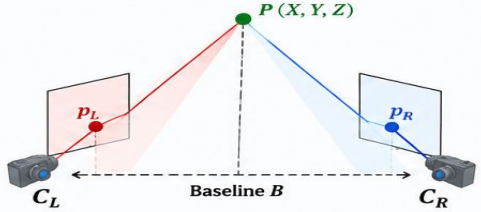
The 3D point is estimated at the intersection of the two rays (or the closest point between them in the presence of noise).



In practice, the rays may not intersect exactly due to noise. We find the closest point between the two rays (e.g., using linear least squares).

4 Recover the 3D coordinates

The triangulated point gives the 3D position of the physical point in the chosen coordinate system (e.g., left camera frame).



Estimated 3D point:

$$P = (X, Y, Z)$$

For a rectified stereo setup, depth can also be computed from disparity:

$$Z = \frac{f \cdot B}{d}$$

The math, kept simple

Linear triangulation (DLT) — each view gives two equations:

```
u · (p₃ᵀX) - p₁ᵀX = 0
v · (p₃ᵀX) - p₂ᵀX = 0

same for view 2 → stack:
A X = 0,  A is 4 × 4
X = last singular vector of A
```

p_1^T, p_2^T, p_3^T are the rows of the camera matrix P; $X = (X, Y, Z, 1)$.

Rectified stereo shortcut — no SVD needed:

$$Z = f \cdot B \,/\, d$$

$$X = (u - cx) \cdot Z \,/\, f$$

$$Y = (v - cy) \cdot Z \,/\, f$$

```
cv2.triangulatePoints(P1, P2, x1, x2)
```

MODULE 4

Stereo Vision

Goal — one camera loses depth. Two cameras recover it.

04

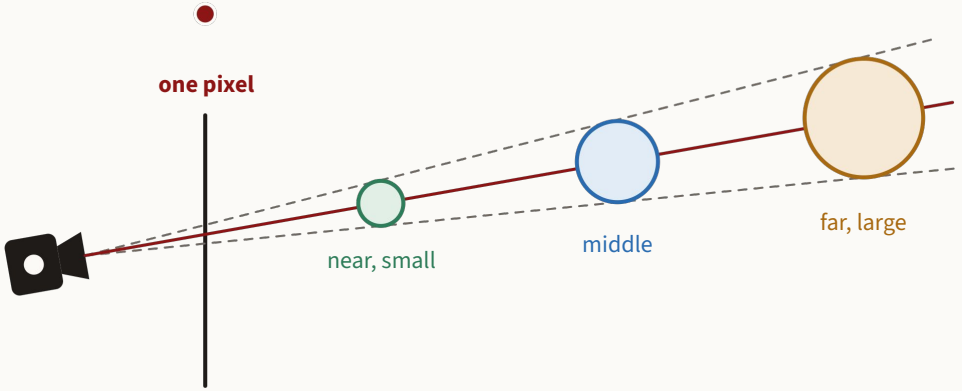
Why 3D | Projection | Calibration | Homography | **Stereo** | Epipolar | Triangulation | Pipeline

Why a single camera fails

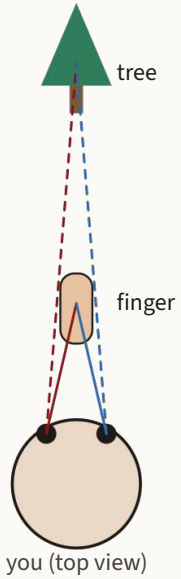
One pixel = one ray. Depth along the ray is lost.

- Every point on the red ray lands on the same pixel
- Small-and-near looks exactly like big-and-far
- No single-image geometry can undo this

> **Think**
> How do monocular depth networks still predict depth? (Hint: learned priors about object sizes and scenes.)

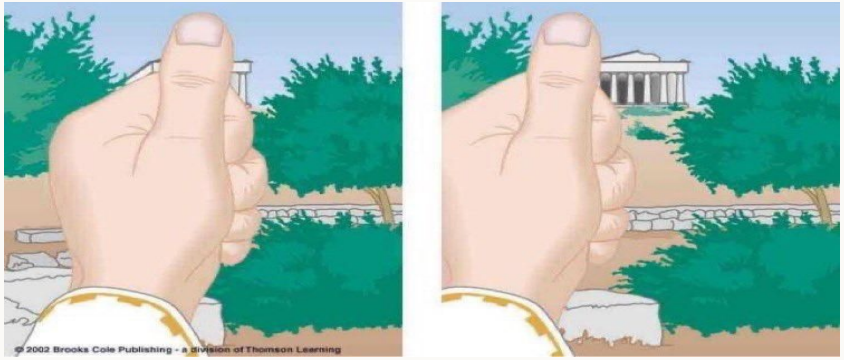


Two eyes: the original stereo system



What the left eye sees

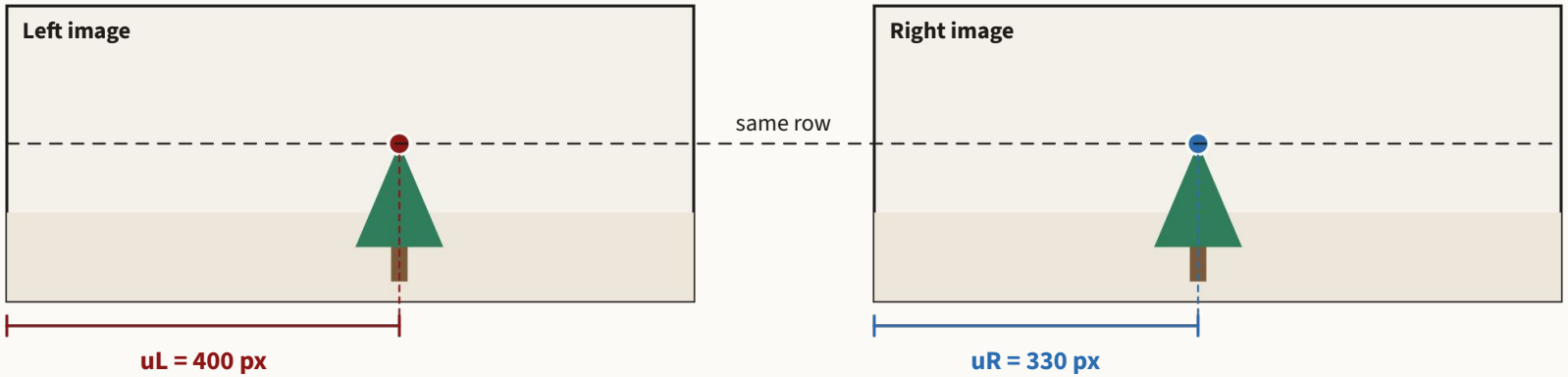
What the right eye sees



finger jumps a lot · tree barely moves

Try it: hold up a finger, close one eye, then the other. That jump is **disparity**.

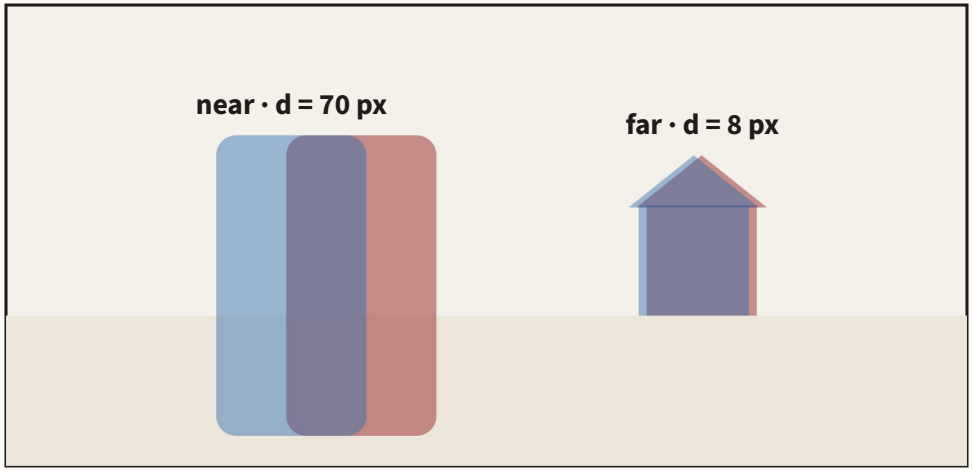
Disparity: the horizontal shift



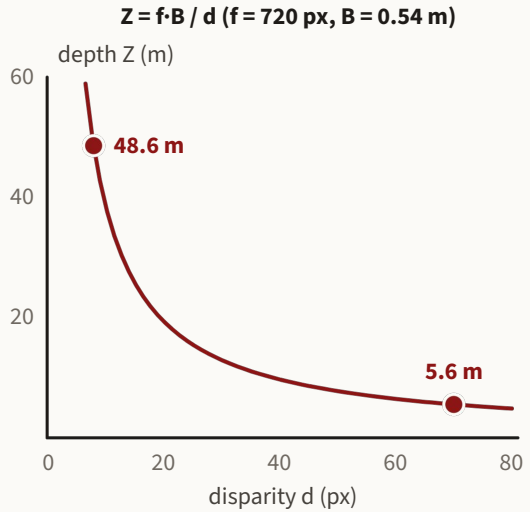
$$d = u\mathrm{L} - u\mathrm{R} = 400 - 330 = \mathbf{70\ px}$$

After rectification the match is on the **same row**. The only unknown left is a horizontal shift: the **disparity**.

Near objects shift more



red = left image · blue = right image, overlaid

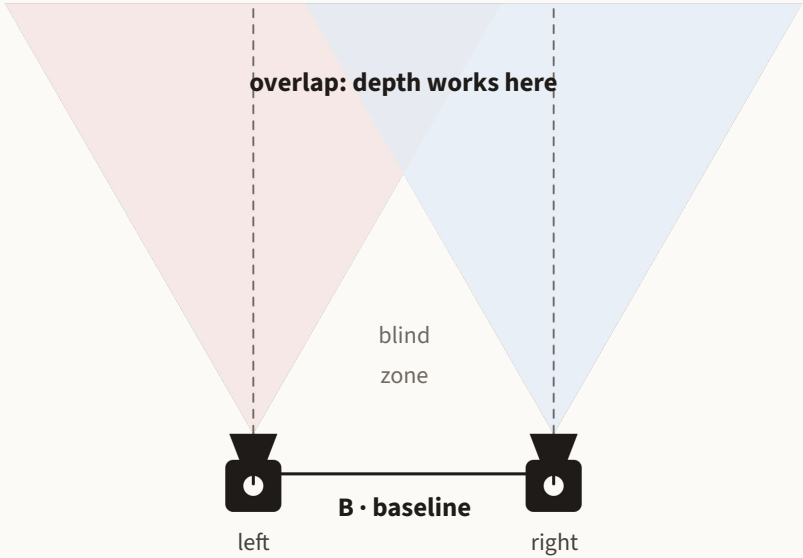


Stereo cameras and the baseline

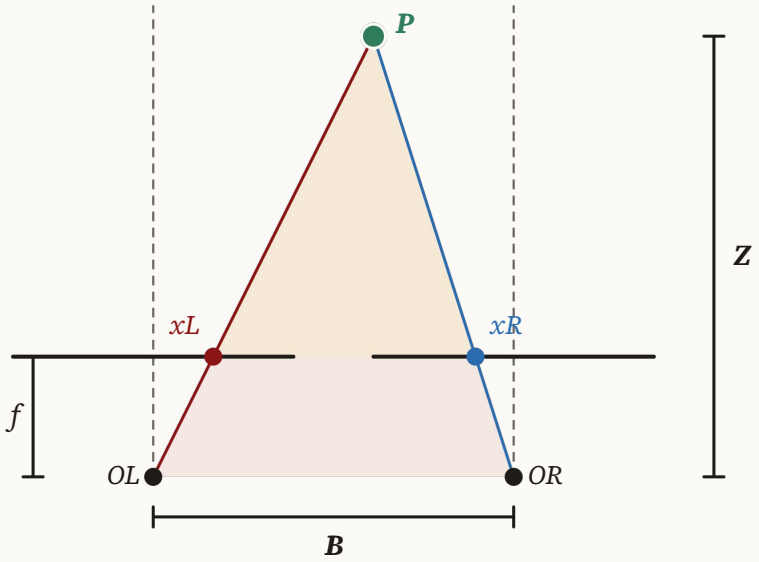
Two cameras, same orientation, side by side.

- **Baseline B** = distance between the optical centers
- Human eyes: B ≈ 6.5 cm
- A car stereo rig: tens of centimeters (KITTI: ≈ 54 cm)

> **Trade-off**
> Wider B → better depth far away, but a bigger blind zone and harder matching.



The depth equation



D = disparity (xL -xR)

$$Z = f \cdot B / d$$

Triangle *P–OL–OR* ~ triangle *P–xL–xR* (similarity rule)

$B / Z = (B - d) / (Z - f) \Rightarrow Z = fB / d$

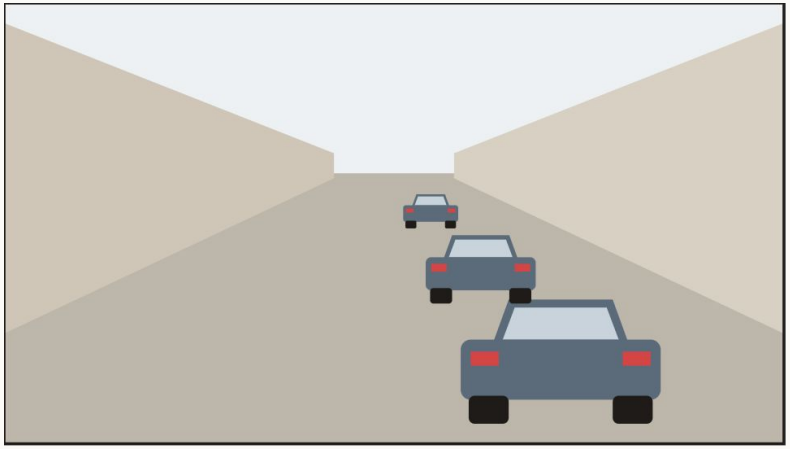
Worked example

f = 720 px, B = 0.54 m, d = 24 px → Z = 720 × 0.54 / 24 = **16.2 m**

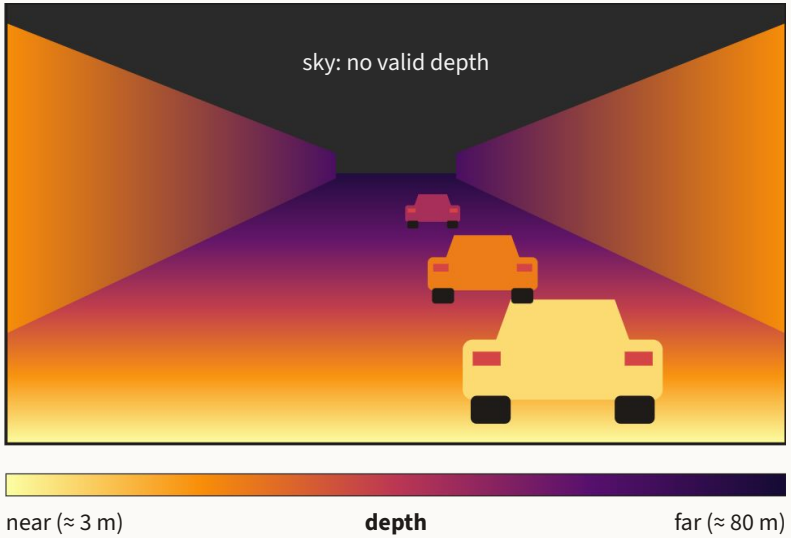
Off by 1 px (d = 23) → 16.9 m. Error grows like Z^2.



Depth map: one distance per pixel

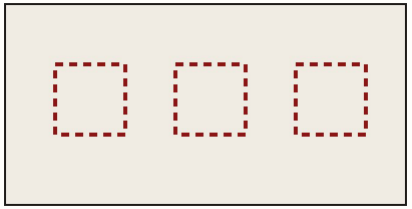


Input: left image



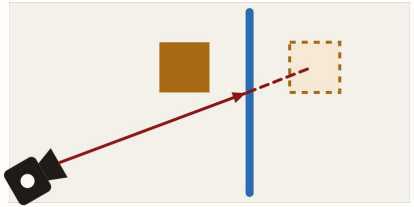
Compute d for every pixel, apply $Z = fB/d$, and color by distance.

Where stereo struggles



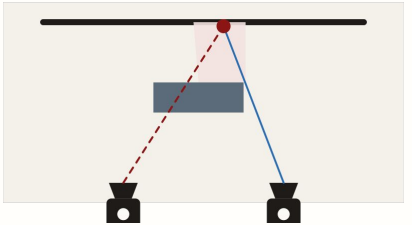
Textureless areas

White walls, sky, road: every patch looks the same, so any match is equally good.



Reflections & glass

The mirror image seems to be behind the glass → wrong, too-far depth.



Occlusions

Some points are seen by only one camera — they have no match at all.

Stereo is only as good as the matching. Next: make matching easier.

MODULE 4

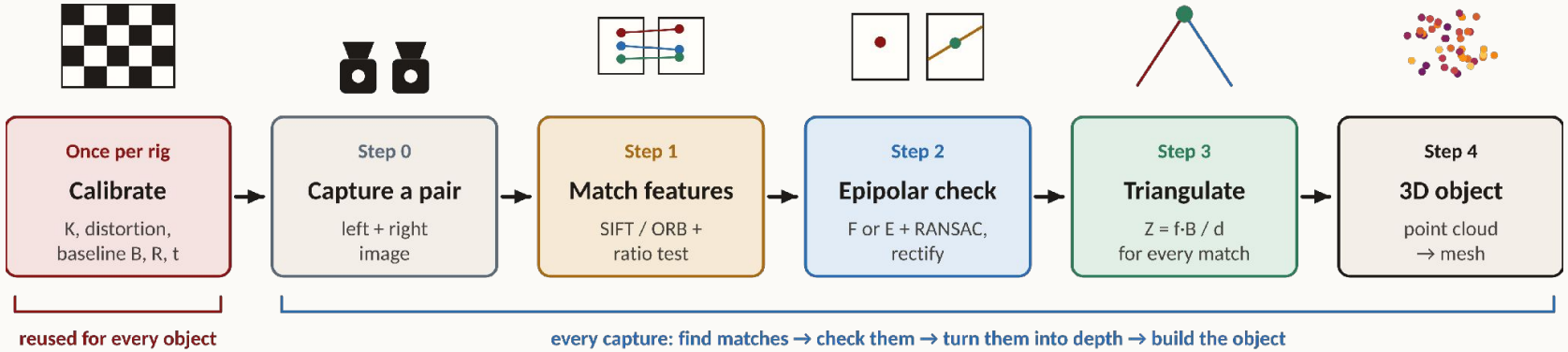
3D Reconstruction

Putting all together to go from 2d to 3d

04

Why 3D | Projection | Calibration | Homography | **Stereo** | Epipolar | Triangulation | Pipeline

3D reconstruction with stereo: the whole flow



In

Two photos of the object taken at the same moment by a calibrated stereo rig: known K, baseline B, and R, t between the cameras.

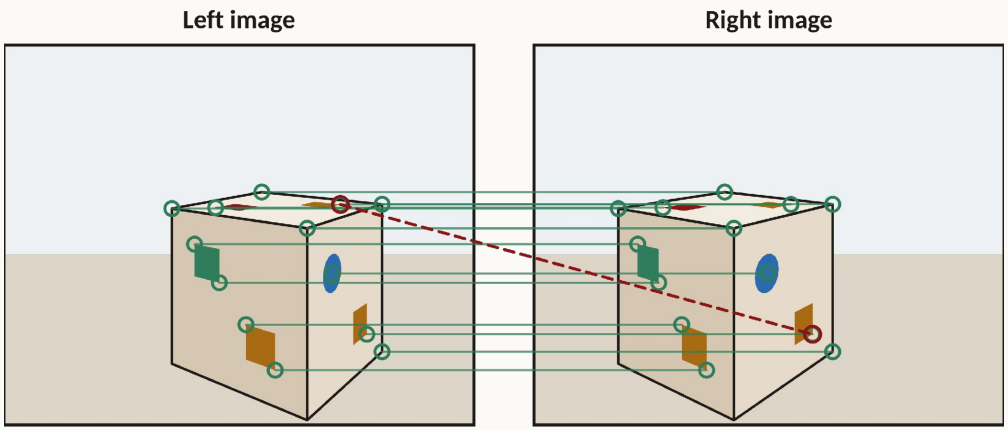
Core idea

Find the same point in both views, then intersect the two viewing rays. For a rectified rig that reduces to $Z = f \cdot B / d$.

Out

A 3D point cloud of the object in meters, which is cleaned and turned into a surface mesh.

Find the same point in both images



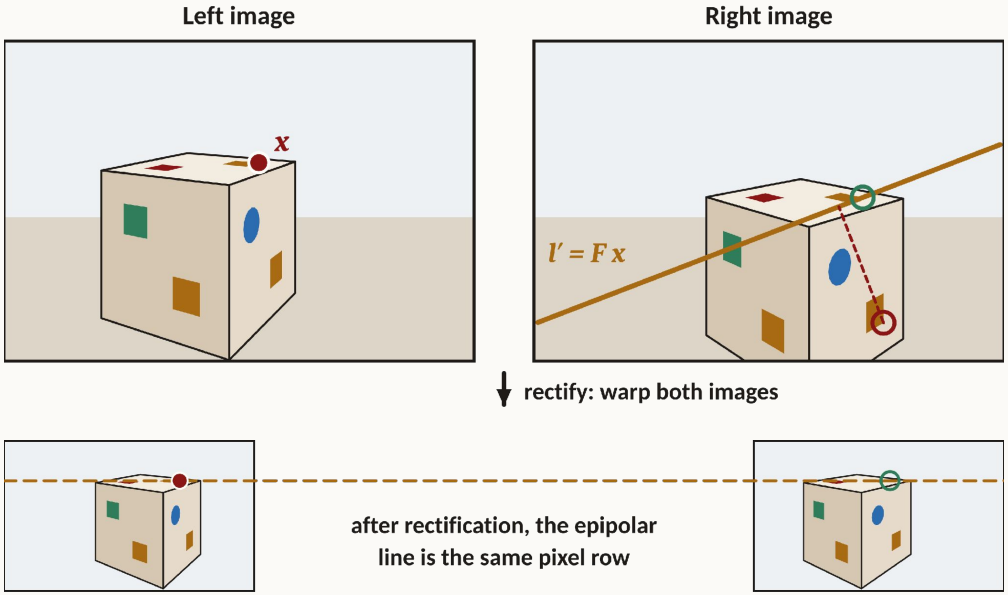
1. **Detect.** Find corners and blobs that are easy to find again (SIFT, ORB).
2. **Describe.** Summarize the patch around each keypoint as a vector (SIFT: 128 numbers).
3. **Match.** Take the nearest vector in the other image, but keep it only if it beats the 2nd best clearly (Lowe ratio < 0.75).

```
sift = cv2.SIFT_create()
kp1, d1 = sift.detectAndCompute(L, None)
m = cv2.BFMatcher().knnMatch(d1, d2, k=2)
```

Result

Hundreds of matches, but look-alike texture still gives a few wrong ones (red). Step 2 removes them.

Epipolar geometry: keep only matches that fit



$$x'^{T} F x = 0$$

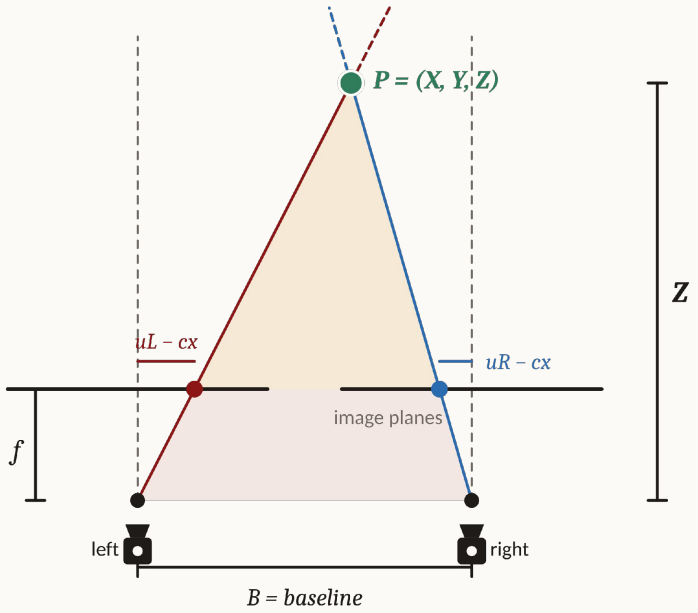
- The match of x must lie on its epipolar line l′ = F x (gold).
- RANSAC: fit F to 8 random matches, count those within ~1 px of their line; repeat, keep the best.
- Matches far from their line are dropped (red).
- Rectify: every epipolar line becomes a pixel row, so dense matching is a 1D search.

```
F, mask = cv2.findFundamentalMat(
    pL, pR, cv2.FM_RANSAC, 1.0)
```

Rig or moving camera?

Rig: R, t come from stereo calibration. Moving camera: estimate E and recover R, t (cv2.recoverPose), with t only up to scale.

Triangulate: from a match to a 3D point



$$d = uL - uR \qquad Z = f \cdot B / d$$
$$X = (uL - cx) \cdot Z / f \qquad Y = (v - cy) \cdot Z / f$$

Worked example

f = 800 px, B = 0.10 m, cx = 640, cy = 360, uL = 520, uR = 480, v = 300
d = 40 px → Z = 800 × 0.10 / 40 = 2.00 m, X = −0.30 m, Y = −0.15 m

How accurate?

$\Delta Z \approx Z^2 / (f \cdot B) \cdot \Delta d$. A 1-pixel error costs 5 cm at 2 m but 20 cm at 4 m. A wider baseline or longer focal length helps.

```
# not rectified? intersect the two rays with DLT
X = cv2.triangulatePoints(P1, P2, xL, xR)
```

STEREO 3D RECONSTRUCTION · STEP 4

From depth to a 3D object



Dense depth map
a disparity for (almost) every pixel

Raw point cloud
every pixel → (X, Y, Z); floor + noise

Clean object points
remove floor, stray and invalid points

Surface mesh
connect the points into triangles

Go dense

Sparse matches only give corners. Run StereoSGBM on the rectified pair for a disparity at nearly every pixel, then cv2.reprojectImageTo3D(disp, Q) gives (X, Y, Z) per pixel.

Clean, then mesh

Drop invalid disparities, remove statistical outliers, and cut away the floor with a RANSAC plane fit. Then build a surface, e.g. Poisson reconstruction in Open3D.

All sides of the object

One stereo pair sees one side. Capture from several viewpoints, align the clouds (ICP) or solve all poses with SfM, and fuse them into one model.